

Optimal Control

First Delivery

Students:

Gonçalo Tomé
Gustavo Rosa
Miguel Martins
Miguel Pimentel

March 2026

Contents

1 Nonlinear Model of a Quadcopter **3**

Glossário

Referenciais e estados do modelo do quadcopter

Símbolo	Nome	Descrição
$\{O_I, X_I Y_I Z_I\}$	Referencial inercial	Fixado à Terra.
$\{O_n, X_n Y_n Z_n\}$	Referencial do quadcopter	Origem no centro de massa, eixos principais.
R_n	Matriz de rotação corpo–inercial	Orientação do quadcopter em relação ao frame inercial
$\mathbf{v}_n = [v_1, v_2, v_3]^\top$	Velocidade linear no corpo	Velocidade do CM no frame do corpo.
$\boldsymbol{\omega}_n = [\omega_1, \omega_2, \omega_3]^\top$	Velocidade angular	Angular do quadcopter no frame do corpo.
$\dot{\mathbf{v}}_n$	Aceleração linear	Derivada de $\mathbf{v}_n$.
$\dot{\boldsymbol{\omega}}_n$	Aceleração angular	Derivada de $\boldsymbol{\omega}_n$.
$\mathbf{V}_n = [\mathbf{v}_n^\top \ \boldsymbol{\omega}_n^\top]^\top$	Velocidade generalizada	Velocidades linear e angular empilhadas.
$\dot{\mathbf{V}}_n$	Aceleração generalizada	Usada em $\dot{\mathbf{V}}_n = M_r^{-1}(\cdots)$.
$\mathbf{r}_n = [r_1, r_2, r_3]^\top$	Posição do quadcopter	Posição do CM no frame inercial.
$\dot{\mathbf{r}}_n$	Velocidade da posição	$\dot{\mathbf{r}}_n = R_n \mathbf{v}_n$.

Parâmetros físicos e inerciais

m_n	Massa do quadcopter	Massa total.
$\mathbf{J}_n = \mathrm{diag}(J_{11}, J_{22}, J_{33})$	Tensor de inércia	Na base dos eixos principais do corpo.
J_{11}, J_{22}, J_{33}	Momentos principais de inércia	Em torno de X_n, Y_n, Z_n.
g	Aceleração da gravidade	Escalar $\approx 9.81\ \mathrm{m/s^2}$.
$\mathbf{g}$	Vetor gravidade (inercial)	$[0, 0, -g]^\top$.
$\mathbf{g}_n$	Gravidade no frame do corpo	$\mathbf{g}_n = R_n^\top \mathbf{g}$.

Forças, momentos e entradas do quadcopter

T_i	Empuxo do rotor i	Força ao longo $+Z_n$, $T_i = K_T \Omega_i^2$.
τ_i	Momento de reação do rotor i	Torque em torno de Z_n, $\tau_i = K_\tau \Omega_i^2$.
F_1, F_2, F_3	Componentes da força no corpo	Resultante das forças dos 4 rotores no frame do corpo
M_1, M_2, M_3	Componentes do momento externo	Momentos resultantes (roll, pitch, yaw).
L	Comprimento do braço	Distância do centro ao rotor.
K_T	Constante de empuxo	Relaciona Ω_i^2 com T_i.
K_τ	Constante de torque	Relaciona Ω_i^2 com τ_i.
Ω_i	Velocidade angular do rotor i	Em rad/s.
u_i	Comando do motor i	Sinal de entrada (0 a $u_{i\,\max}$).
$K_{u\to\Omega^2}$	Ganho entrada–Ω^2	$\Omega_i^2 = K_{u\to\Omega^2} u_i$.
$u_{i\,\max}$	Saturação do comando	Máximo valor de u_i.

Matrizes principais do modelo rigid body

$\mathbf{M}_r$	Matriz de massa rígida	$\begin{bmatrix} m_n I & 0 \\ 0 & \mathbf{J}_n \end{bmatrix}$.
$\mathbf{C}_{rn}(\boldsymbol{\omega}_n)$	Matriz não linear inercial	Parâmetros de massa/inércia de Forças de Coriolis, centrífugos e giroscópicos (momentos).
$\mathbf{F}_n^{(\mathrm{gen})}$	Forças generalizadas	$[\mathbf{F}_n^\top \ \mathbf{M}_n^\top]^\top$.
$\mathbf{G}_{rn}$	Forças gravíticas generalizadas	$[m_n \mathbf{g}_n^\top \ 0^\top]^\top$.

Símbolos usados na linearização do modelo

$\mathbf{v}_n', \boldsymbol{\omega}_n'$	Variações de velocidade	Pequenas variações em torno do equilíbrio.
F_3', M_1', M_2', M_3'	Variações de entrada	Desvios em relação ao ponto de equilíbrio.
$\theta_1', \theta_2', \theta_3'$	Pequenos ângulos	Roll, pitch, yaw lineares.
$\mathbf{x}$	Vetor de estado linearizado	Empilha $r_i, \dot{r}_i', \theta_i', \dot{\theta}_i'$.
$\mathbf{u}$	Vetor de entrada linearizado	$[F_3' \ M_1' \ M_2' \ M_3']^\top$.
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	Matrizes de estado	Modelo $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$.
$H_1(s) \ldots H_4(s)$	Funções de transferência	Relações entrada–saída para cada eixo.

1 Nonlinear Model of a Quadcopter

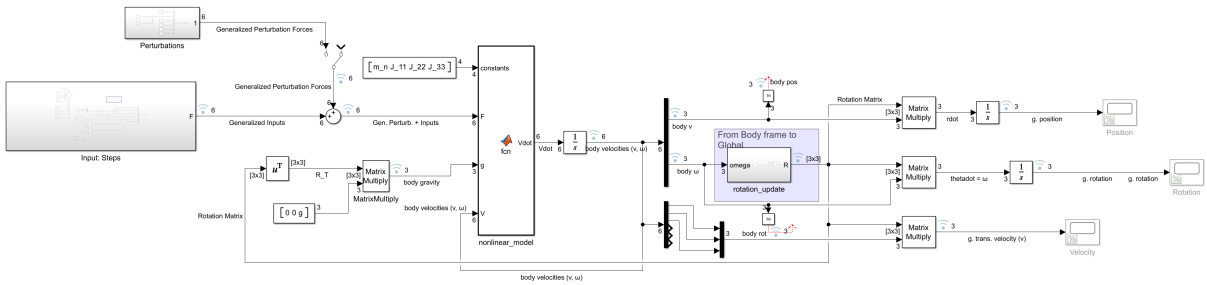


Figure 1: Nonlinear Model's Simulink

1.

There are four main vectors that make up the state vector: position (r_1,r_2,r_3); velocity (v_1,v_2,v_3); orientation (θ_1,θ_2,θ_3); angular velocity (ω_1,ω_2,ω_3).

The orientation variables θ_i appear in the linearized (first order) version of the matrix R_n (R_n). As such, the state vector is presented below.

$$x = [v_1, v_2, v_3, \omega_1, \omega_2, \omega_3, r_1, r_2, r_3, \theta_1, \theta_2, \theta_3]^T \tag{1}$$

As for the input vector, it is as presented below:

$$[F_3, M_1, M_2, M_3]^T \tag{2}$$

And the external force perturbations plug:

$$[F_{ext,1}; F_{ext,2}; F_{ext,3}]^T \tag{3}$$

2.

If the drone is at rest in the air $\Rightarrow v_n = 0, \omega_n = 0, \dot{v_n} = 0, \dot{\omega}_n = 0$, for each n in each of the 3 dimensions.

In [1], the z axis points downward and, as such, the gravity acceleration vector is:

$$g_n = R_n^T g \cdot [0; 0; g]^T \tag{4}$$

From eq. 2.5, [1]:

$$m_n(\dot{v_2} + v_2\omega_1 + v_1\omega_1) = F_3 + m_n g_3 \Leftrightarrow F_3 = -m_n g = -0.618N \tag{5}$$

Since this force is equally divided between each of the four motors:

$$u_1 = u_2 = u_3 = u_4 = \frac{m_n g}{4K_{u\to\Omega^2}K_T} = 236.8 \tag{6}$$

It should be noted that this value is smaller than the saturation value for the motors, $236.8 < 500 = u_{i,max}$

3.

Starting from hover and applying a step :

$$F_3 = -m_n g_3 + \Delta F_3, \Delta F_3 < 0 \tag{7}$$

Since the drone is at rest at the starting instant, v_n, ω_n and θ_n are equal. With this in consideration, we can say:

From Eq. 2.3 and 2.4 [1] (Horizontal):

$$m_n(\dot{v_1} + v_3\omega_2 - v_2\omega_3) = F_1 + m_n g_1 \Rightarrow \dot{v}_1 = 0 \tag{8}$$

$$m_n(\dot{v_2} + v_1\omega_3 - v_3\omega_1) = F_2 + m_n g_2 \Rightarrow \dot{v}_2 = 0 \tag{9}$$

From Eq. 2.5 [1] (Vertical):

$$m_n(\dot{v_3} + v_2\omega_1 - v_1\omega_2) = F_3 + m_n g_3 \Rightarrow \dot{v}_3 = m_n g_3 - m_n g_3 + \Delta F_3 \Leftrightarrow m_n \dot{v}_3 = \Delta F_3 \tag{10}$$

From Eq. 2.6-2,8 [1] (angular), written in index form:

$$J_{ii}\dot{\omega}_i + (J_{ii} - J_{kk})\omega_j\omega_k = M_i = 0 \Rightarrow \dot{\omega}_1 = \dot{\omega}_2 = \dot{\omega}_3 = 0 \tag{11}$$

Taking the expression extracted from Eq. 2.5 [1], (since $\dot{v_3}$ increases as a step function) and integrating it with respect to time, we conclude that v_3 increases linearly. Integrating it once more we conclude that r_3 increases parabolically.

The model behaves as expected, it accelerates upwards from a net upward force ΔF_3, all other state variables remain the same, because there are no lateral forces, no net moments in relation with the center of mass of the drone and no angular acceleration.

The only system that is excited is the vertical subsystem $H_1(s) = \frac{1}{m_s s^2}$ (eq 2.56 [1]) that behaves as a double integrator.

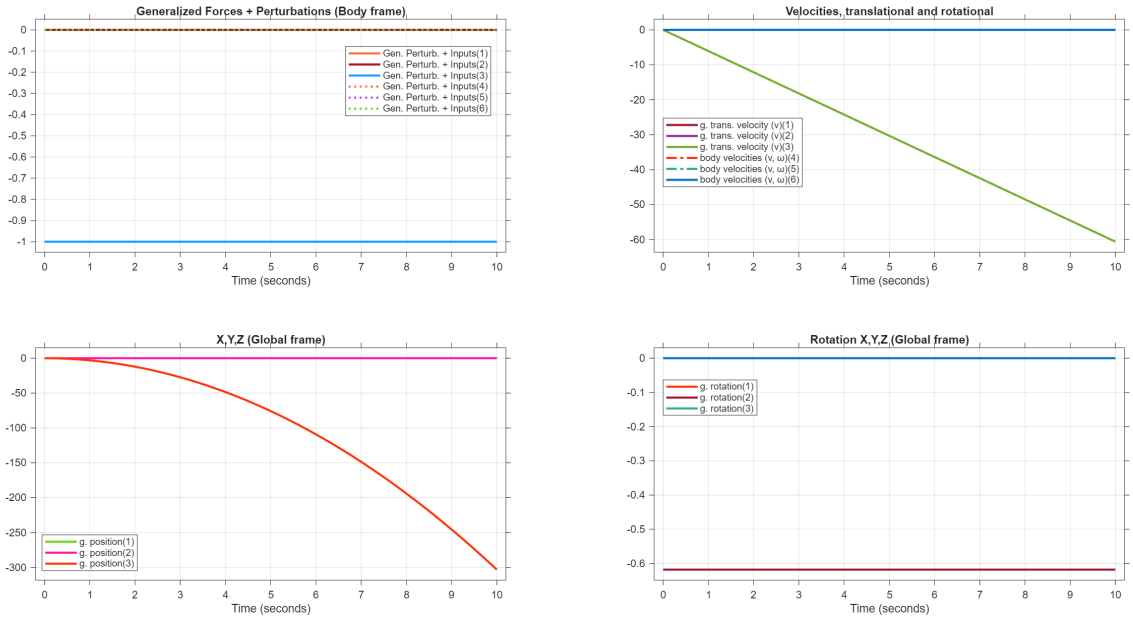


Figure 2: Simulink results for ex. 3. Step value of: $\Delta F_3 = -0.382N$

4.

While hovering and applying a step in M3:

$$M_3 = \lambda(t), t > t_i : \lambda(t) = step\ value = a \quad , \quad except : 0 \tag{12}$$

It is expected the model to only rotate the drone by itself. In other words: $\theta_3(t) \neq 0$ and $F_3 = -m_n g_3$ and the rest is 0.

Since the drone is hovering it means that he is still in a position, having to consider once again v_n, w_n and θ_n = 0 and F_1, F_2, M_1, M_2 = 0:

$$M_1 = J_{11}\,\dot{\omega}_1 + (J_{33} - J_{22})\,\omega_2\,\omega_3 \Rightarrow \dot{\omega}_1 = 0 \tag{13}$$

$$M_2 = J_{22}\,\dot{\omega}_2 + (J_{11} - J_{33})\,\omega_1\,\omega_3 \Rightarrow \dot{\omega}_2 = 0 \tag{14}$$

$$M_3 = J_{33}\,\dot{\omega}_3 + (J_{22} - J_{11})\,\omega_1\,\omega_2 \Rightarrow \dot{w}_3 = \lambda(t)/J_{33} \Rightarrow w_3 = a(t_f - t_i)/J_{22} \tag{15}$$

$$m_n(\dot{v}_1 + v_3\omega_2 - v_2\omega_3) = F_1 + m_n g_1 \Rightarrow \dot{v}_1 = 0 \tag{16}$$

$$m_n(\dot{v}_2 + v_1\omega_3 - v_3\omega_1) = F_2 + m_n g_2 \Rightarrow \dot{v}_2 = 0 \tag{17}$$

$$m_n(\dot{v}_3 + v_2\omega_1 - v_1\omega_2) = F_3 + m_n g_3 \Rightarrow \dot{v}_3 = 0 \tag{18}$$

As expected, the drone will only have rotation in its third axis due to only having $\omega_3 \neq 0$.

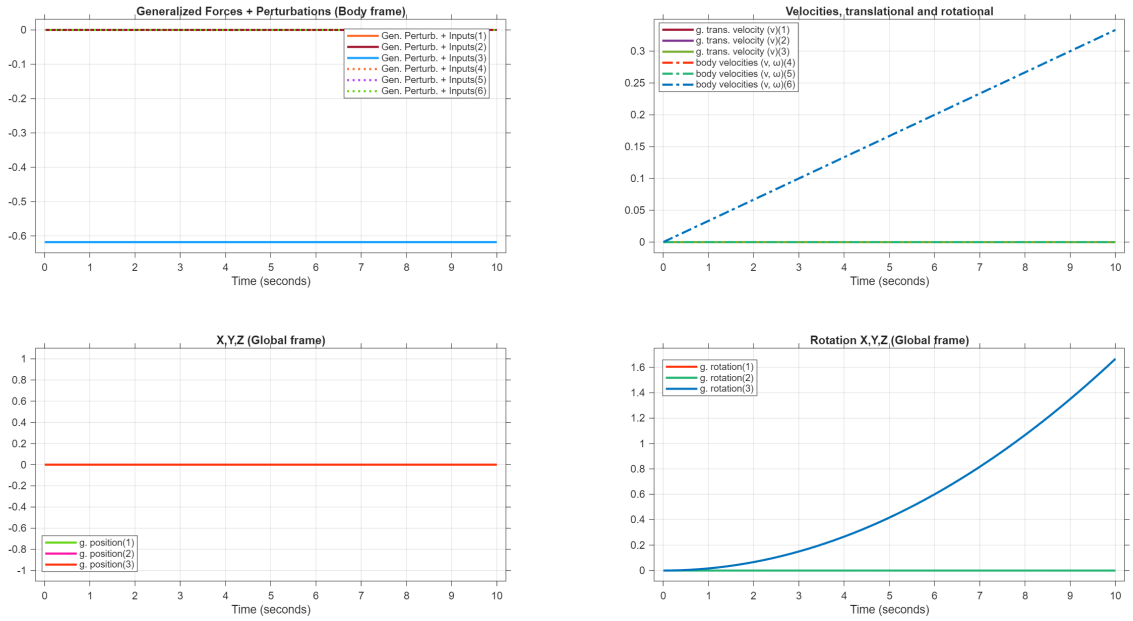


Figure 3: Simulink results for ex. 4. Step value of: $a = 1 \times 10^{-6} N \cdot m$

5.

Applying just a step in M_2 and taking M_1,M_3 = 0, $F_n = 0$ and $w_1, w_3 = 0$, (Note that gravity only depends in position, not time.):

$$M_2 = \lambda(t), t > t_i : \lambda(t) = step\ value = a \quad , \quad except : 0 \tag{19}$$

$$M_2 = J_{22}\,\dot{\omega}_2 + (J_{11} - J_{33})\,\omega_1\,\omega_3 \Rightarrow \dot{\omega_2} = \lambda(t)/J_{22} \Rightarrow \omega_2 = a * t/J_{22},\ t_i < t \tag{20}$$

$$M_1 = J_{11}\,\dot{\omega}_1 + (J_{33} - J_{22})\,\omega_2\,\omega_3 \Rightarrow \dot{\omega_1} = 0 \tag{21}$$

$$M_3 = J_{33}\,\dot{\omega}_3 + (J_{22} - J_{11})\,\omega_1\,\omega_2 \Rightarrow \dot{\omega_3} = 0 \tag{22}$$

$$m_n(\dot{v_1}+v_3\omega_2-v_2\omega_3) = F_1+m_ng_1 \Rightarrow \dot{v_1} = g_1+v_3*(a*t)/J_{22} \approx g_1 \Rightarrow v_1 = g_1*t\,,\ for\,time \approx< 8min \tag{23}$$

$$m_n(\dot{v_2} + v_1\omega_3 - v_3\omega_1) = F_2 + m_ng_2 \Rightarrow \dot{v_2} = g_2 \Rightarrow v_2 = g_2 * t \tag{24}$$

$$m_n(\dot{v_3}+v_2\omega_1-v_1\omega_2) = F_3+m_ng_3 \Rightarrow \dot{v_3} = g_3+v_1*(a*t)/J_{22} \approx g_3 \Rightarrow v_3 = g_3*t,\ for\,time \approx< 8min \tag{25}$$

With the rotation + gravity only actuating it can be concluded that with very high rotations and with high velocities $r_{1\ global}$ will start going towards negative values (around $time \approx< 8min$ with a step value of $a = 1 * 10^{-6}Nm$, Figure 4).

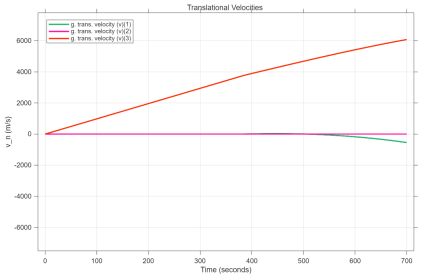


Figure 4: 700 second Global Position output

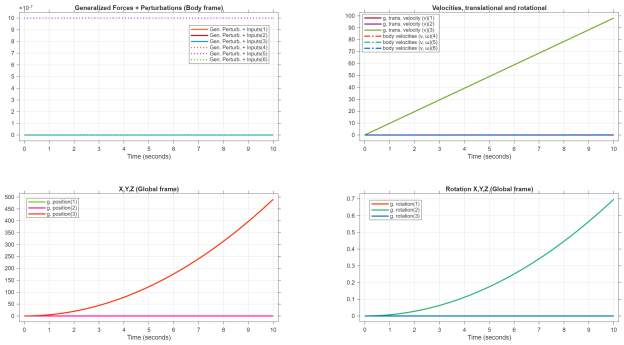


Figure 5: Simulink results for ex. 5

6.

In order to apply a lateral perturbation while hovering it is needed to apply in the body frame a temporary value of F_2 (in the y axis of the body). For this it was added two steps, one starting at 1 second with $0.5N$ and the other starting at 2 seconds with $-0.5N$ in order to cancel each other. Since there is no closed loop in order to correct position/angle it is expected that the drone does not stop going towards positive values of y. Take note that the model is

not modeling the friction so the drone will not stop just like in the void/space for example. If it was needed to model some friction it could be done by adding another force in the other directions seconds after the perturbation and is expected that after some time the drone will stop moving in y.

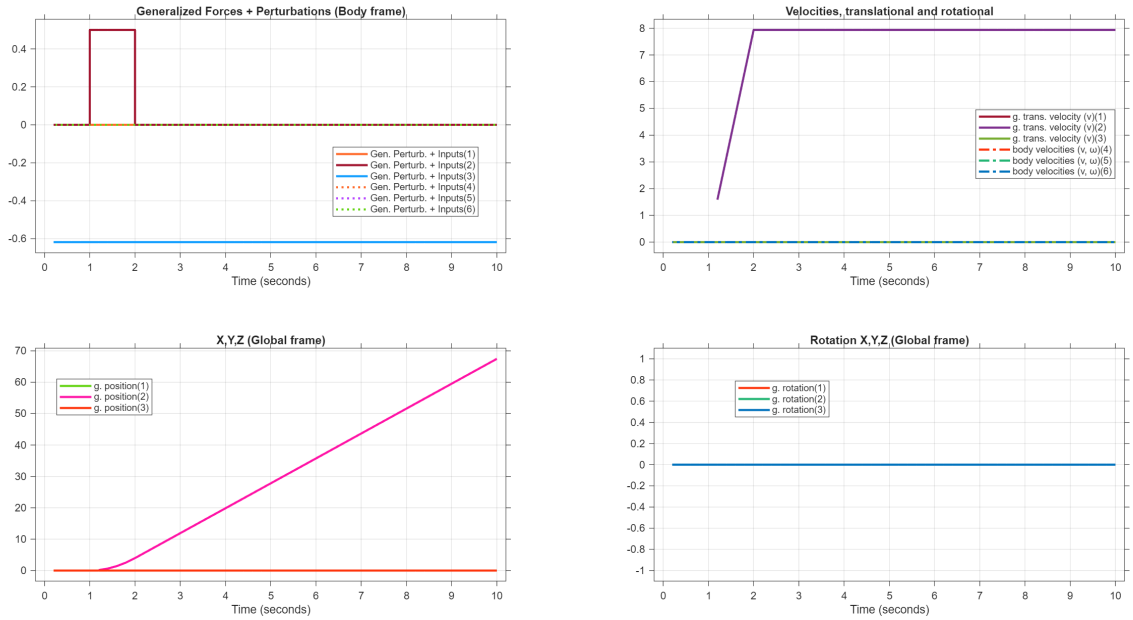


Figure 6: Simulink results for ex. 6.
The translational velocity in y (purple line) should start at t=1s with $v_2 = 0$. For some reason MATLAB didn't plot it right.

7.

To take off, move while rotating and landing with a rotation of π $(180°)$ an input of various blocks was needed to simulate this behavior. Multiple ramp functions were implemented in F_3 along the time adding and subtracting each other in order to simulate the green velocity. The M_3 was modeled with step inputs to simulate the rotation.

The ramp functions values for F_3 are completely arbitrary as long as they are all equal in value. For the M_3 some calculations were necessary in order to perfectly achieve the $90°$ in middle of the flight and consequently $180°$ landed:

$$M_3 = J_{33}\dot{\omega_3} \Rightarrow \omega_3 = M_3 \times t/J_{33} \Rightarrow \theta_3 = t^2/2 \ \times \ M_3/J_{33} \tag{26}$$

Trying to find the M_3 step value for $\theta_3 = \pi/2$ at $t = 20s$ gives $M_3 = 2.35619 \times 10^{-7} Nm$.

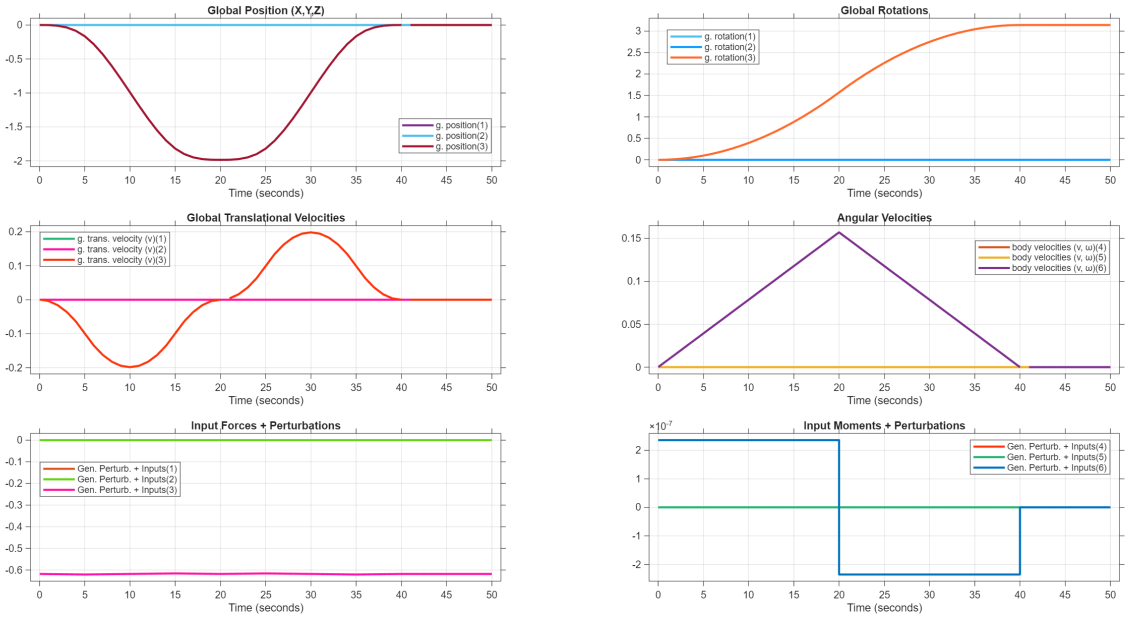


Figure 7: Simulink results for ex. 7.

References

[1] Jorge Martins, Rui Coelho, and André Carvalho. *Rigid Body Dynamics: Modeling and Simulation of a Quadcopter. Optimal Control Course Notes*. Instituto Superior Técnico, University of Lisbon. 2022. URL: https://fenix.tecnico.ulisboa.pt/downloadFile/1407993358945355/quadcopterDynamics.pdf.



Optimal Control

Second Delivery

Students:

Gonçalo Tomé
Gustavo Rosa
Miguel Martins
Miguel Pimentel

May 2026

Contents

1 Kalman-Bucy Observer **3**

2 LQR Servo Controller **11**

Glossary

Reference frames and states of the quadcopter model

Symbol	Name	Description
$\{O_I, X_I Y_I Z_I\}$	Inertial reference frame	Earth-fixed.
$\{O_n, X_n Y_n Z_n\}$	Quadcopter reference frame	Origin at the center of mass, principal axes.
R_n	Body-to-inertial rotation matrix	Orientation of the quadcopter with respect to the inertial frame
$\mathbf{v}_n = [v_1, v_2, v_3]^\top$	Linear velocity in the body frame	Velocity of the CM in the body frame.
$\boldsymbol{\omega}_n = [\omega_1, \omega_2, \omega_3]^\top$	Angular velocity	Quadcopter angular velocity in the body frame.
$\dot{\mathbf{v}}_n$	Linear acceleration	Derivative of $\mathbf{v}_n$.
$\dot{\boldsymbol{\omega}}_n$	Angular acceleration	Derivative of $\boldsymbol{\omega}_n$.
$\mathbf{V}_n = [\mathbf{v}_n^\top \ \boldsymbol{\omega}_n^\top]^\top$	Generalized velocity	Stacked linear and angular velocities.
$\dot{\mathbf{V}}_n$	Generalized acceleration	Used in $\dot{\mathbf{V}}_n = M_r^{-1}(\cdots)$.
$\mathbf{r}_n = [r_1, r_2, r_3]^\top$	Quadcopter position	Position of the CM in the inertial frame.
$\dot{\mathbf{r}}_n$	Velocity of the position	$\dot{\mathbf{r}}_n = R_n \mathbf{v}_n$.

Physical and inertial parameters

m_n	Quadcopter mass	Total mass.
$\mathbf{J}_n = \mathrm{diag}(J_{11}, J_{22}, J_{33})$	Inertia tensor	In the basis of the body's principal axes.
J_{11}, J_{22}, J_{33}	Principal moments of inertia	About X_n, Y_n, Z_n.
g	Gravitational acceleration	Scalar $\approx 9.81\ \mathrm{m/s^2}$.
$\mathbf{g}$	Gravity vector (inertial)	$[0, 0, -g]^\top$.
$\mathbf{g}_n$	Gravity in the body frame	$\mathbf{g}_n = R_n^\top \mathbf{g}$.

Forces, moments, and quadcopter inputs

T_i	Thrust of rotor i	Force along $+Z_n$, $T_i = K_T \Omega_i^2$.
τ_i	Reaction moment of rotor i	Torque about Z_n, $\tau_i = K_\tau \Omega_i^2$.
F_1, F_2, F_3	Body force components	Resultant of the forces from the 4 rotors in the body frame
M_1, M_2, M_3	External moment components	Resultant moments (roll, pitch, yaw).
L	Arm length	Distance from the center to the rotor.
K_T	Thrust constant	Relates Ω_i^2 to T_i.
K_τ	Torque constant	Relates Ω_i^2 to τ_i.
Ω_i	Angular velocity of rotor i	In rad/s.
u_i	Motor i command	Input signal (0 to $u_{i\,\max}$).
$K_{u\to\Omega^2}$	Input-to-Ω^2 gain	$\Omega_i^2 = K_{u\to\Omega^2} u_i$.
$u_{i\,\max}$	Command saturation	Maximum value of u_i.

Main matrices of the rigid body model

$\mathbf{M}_r$	Rigid mass matrix	$\begin{bmatrix} m_n I & 0 \\ 0 & \mathbf{J}_n \end{bmatrix}$.
$\mathbf{C}_{rn}(\boldsymbol{\omega}_n)$	Inertial nonlinear matrix	Mass/inertia parameters of Coriolis, centrifugal, and gyroscopic forces (moments).
$\mathbf{F}_n^{(\mathrm{gen})}$	Generalized forces	$[\mathbf{F}_n^\top \ \mathbf{M}_n^\top]^\top$.
$\mathbf{G}_{rn}$	Generalized gravitational forces	$[m_n \mathbf{g}_n^\top \ 0^\top]^\top$.

Symbols used in the model linearization

$\mathbf{v}_n', \boldsymbol{\omega}_n'$	Velocity variations	Small variations around equilibrium.
F_3', M_1', M_2', M_3'	Input variations	Deviations from the equilibrium point.
$\theta_1', \theta_2', \theta_3'$	Small angles	Linear roll, pitch, and yaw.
$\mathbf{x}$	Linearized state vector	Stacks $r_i, \dot{r}_i', \theta_i', \dot{\theta}_i'$.
$\mathbf{u}$	Linearized input vector	$[F_3' \ M_1' \ M_2' \ M_3']^\top$.
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	State matrices	Model $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$.
$H_1(s) \dots H_4(s)$	Transfer functions	Input-output relationships for each axis.

1 Kalman-Bucy Observer

All physical systems are affected by noise. The Kalman-Bucy Observer was created in order to attenuate these disturbances and to provide an optimal estimate of the system states. This observer is a variation from a normal observer, where L, the observer gain block, is adapted to tune in the reading from model simulation and sensor measurements. L_K mixes the*a priori* information (system model) with *a posteriori* info. (measurements) in order to minimize the estimation error.

Before implementing the Kalman-Bucy Observer, the influence of noise on the linearized closed-loop system is analyzed in Exercise 5:

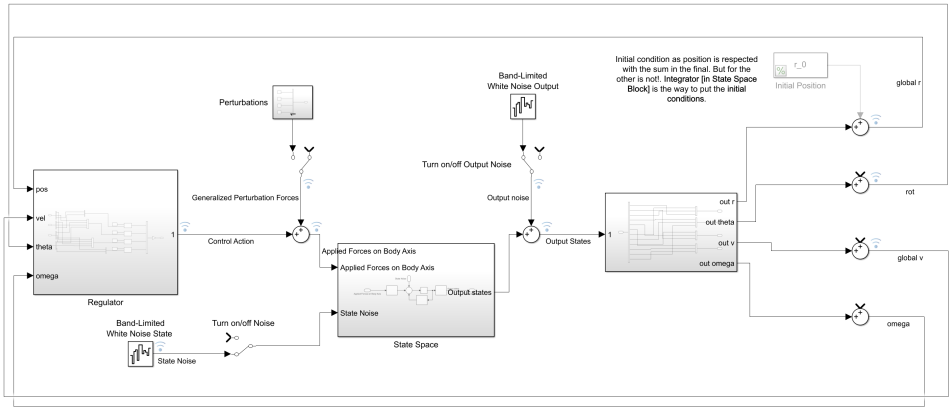


Figure 1: Regulator for Linear Model – Simulink

Table 1: Band-Limited white noise data [2]

State/Output	**State** Noise Variance $\equiv \sigma_P^2$	**Output** Noise Variance $\equiv \sigma_M^2$
Inertial Position X, r_1 (m)	9×10^{-1}	7.3221×10^{-1}
Inertial Position Y, r_2 (m)	9×10^{-1}	4.2821×10^{-1}
Inertial Position Z, r_3 (m)	1×10^{-1}	1.8010×10^{-3}
Roll, θ_1 (rad)	3×10^{-1}	3.4817×10^{-4}
Pitch, θ_2 (rad)	2×10^{-1}	2.6060×10^{-4}
Yaw, θ_3 (rad)	1×10^{-1}	3.8313×10^{-2}
Body angular velocity X, ω_1 (rad/s)	1×10^{-3}	1.8271×10^{-6}
Body angular velocity Y, ω_2 (rad/s)	1×10^{-3}	4.1449×10^{-6}
Body angular velocity Z, ω_3 (rad/s)	1×10^{-3}	7.0775×10^{-6}
Body Linear velocity X, v_1 (m/s)	2×10^{-1}	2.2190×10^{-3}
Body Linear velocity Y, v_2 (m/s)	2×10^{-1}	2.2190×10^{-3}
Body Linear velocity Z, v_3 (m/s)	1×10^{-2}	1.8271×10^{-4}

For this case of study, stochastic Gaussian white noise will be introduced to both state and output measurements. It should be noted, however, that assuming all noise sources are white may not always be appropriate or valid, as white noise is characterized by a constant power spectral density. In practice, many real-world disturbances do not conform to this idealization and should not be approximated as white noise.

1.

Measurement noise can arise from various sources, including sensor deficiencies. For camera-based sensors, specific sources include dirt or other obstructions on the lens, water droplets, poor system calibration, motion artifacts, camera shake, undesirable voltage fluctuations in the motors, and vibrations from the propellers. For accelerometers, measurement noise typically arises from mechanical vibrations transmitted from the motors and propellers.

2.

Process noise encompasses unmodeled dynamics and external disturbances. For the quadcopter, examples include aerodynamic effects (which vary with the angle of the drone), environmental disturbances (wind gusts, turbulence), and structural imperfections like airframe elasticity.

3.

In the Kalman Filter, process noise $w(t)$ and sensor noise $v(t)$ are introduced to the state space model, as shown below:

$$\begin{aligned} \dot{x}(t) &= Ax(t) + Bu(t) + w(t) \\ y(t) &= Cx(t) + v(t) \end{aligned} \tag{1}$$

As mentioned previously, the noise was implemented as gaussian white noise, with zero mean, and variances Q_0 and R_0 for $w(t)$ and $v(t)$, respectively. In practice, due to the multidimensionality of the system, Q_0 and R_0 are not scalar variances but 12×12 covariance matrices, where the diagonal has the variance information for each state/output and the remaining entries give the covariance between the noise affecting different variables. One of the main assumptions of the Kalman filter is that $w(t)$ and $v(t)$ are completely uncorrelated, meaning that Q_0 and R_0 only have values in their diagonal, those values being shown in table 1. In simple terms, they are the matrix version of σ_P^2 and σ_M^2. The values of all other elements depends on the simulation purpose, the relations of noise between each position, orientation and velocity. Note that the Q_0 in this case is equal to $B \times Q_{0-actuators_{[4\times4]}} \times B^T$.

$$Q_0 = E\{w(t)w^T(t)\} \qquad R_0 = E\{v(t)v^T(t)\} \tag{2}$$

$w(t)_{[12\times1]} \equiv$ Stochastic Signal with var. σ_P^2 $\qquad$ $v(t)_{[12\times1]} \equiv$ Stochastic Signal with var. σ_M^2

$E\{\} \equiv$ Expected Value

4.

The relative difference between R_0 and Q_0 affects the bandwidth of the filter. The larger the bandwidth, the more signal information can be captured (at the cost of capturing more noise). The lesser the bandwidth (huge R_0 and tiny Q_0, for example) will not make things better since it makes the system react slower imposing a greater phase lag. An optimal bandwidth between both matrices needs to be found.

An optimal choice of the Q_0 and R_0 matrices is putting all values, except their diagonal, to 0, meaning that the noise sources are independent, uncorrelated. Providing the matrices with this values guarantees that the actuator gain, L_K, will trust the sensors exactly as much as they deserve to be trusted compared to the mathematical model.

5.

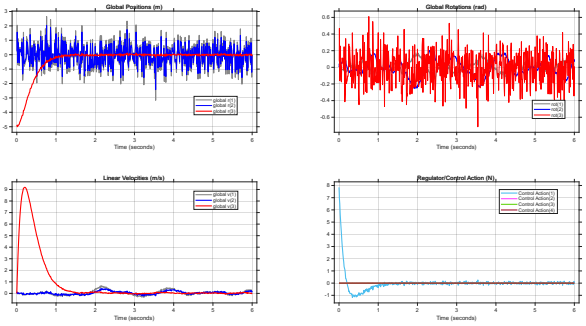


Figure 2: Regulator Linear Model – Initial conditions changed: $r_0 = [0, 0, -5]$ (5m above ground)

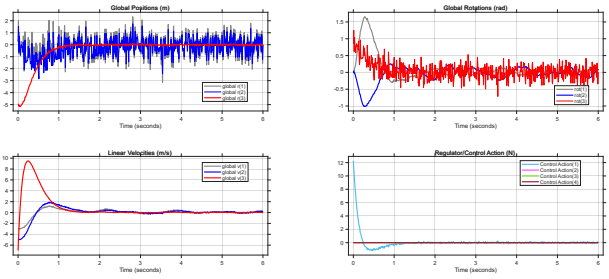


Figure 3: Regulator Linear Model – Initial conditions changed: $r_0 = [0, 0, -5]$, $\theta = [0, 0, \pi/4]$ and $v_0 = [-3, -5, -7, \mathbf{0}]$ (m/s)

Regulator aims to converge all states to zero but the added noise continuously disrupts this stabilization. Consequently, the regulator must act at each time step to reject these stochastic disturbances and drive the system back to its equilibrium point, as illustrated in Figure 3.

The Kalman-Bucy Observer with Regulator

Similarly to a LQR problem, the optimal gains for the Kalman-Bucy observer, L_k are obtained through equation 3, where P is the solution of the algebraic Ricatti equation (equation 4).

$$L_K = PC^T R_0^{-1} \tag{3}$$

$$AP + PA^T + BQ_{0-actuators_{[4\times4]}}B^T - PC^T + R_0^{-1} + CP = 0 \tag{4}$$

This was implemented with MATLAB's function *lqe()* for each decoupled subsystem of the State Space in `workspace_initialization.m`.

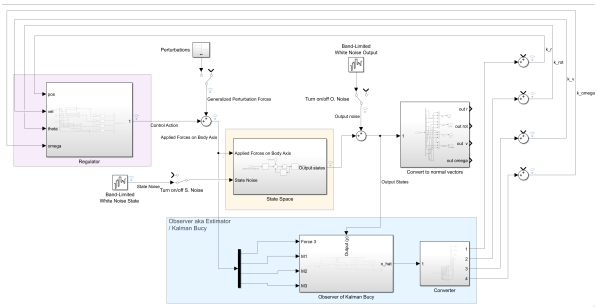


Figure 4: Kalman-Bucy Linear model – Simulink

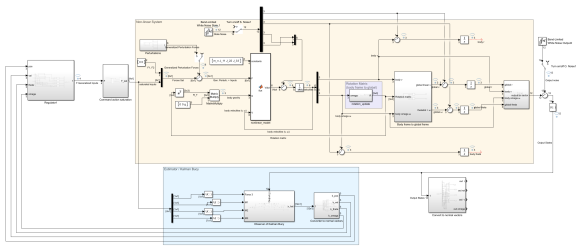


Figure 5: Kalman-Bucy Non-linear model – Simulink

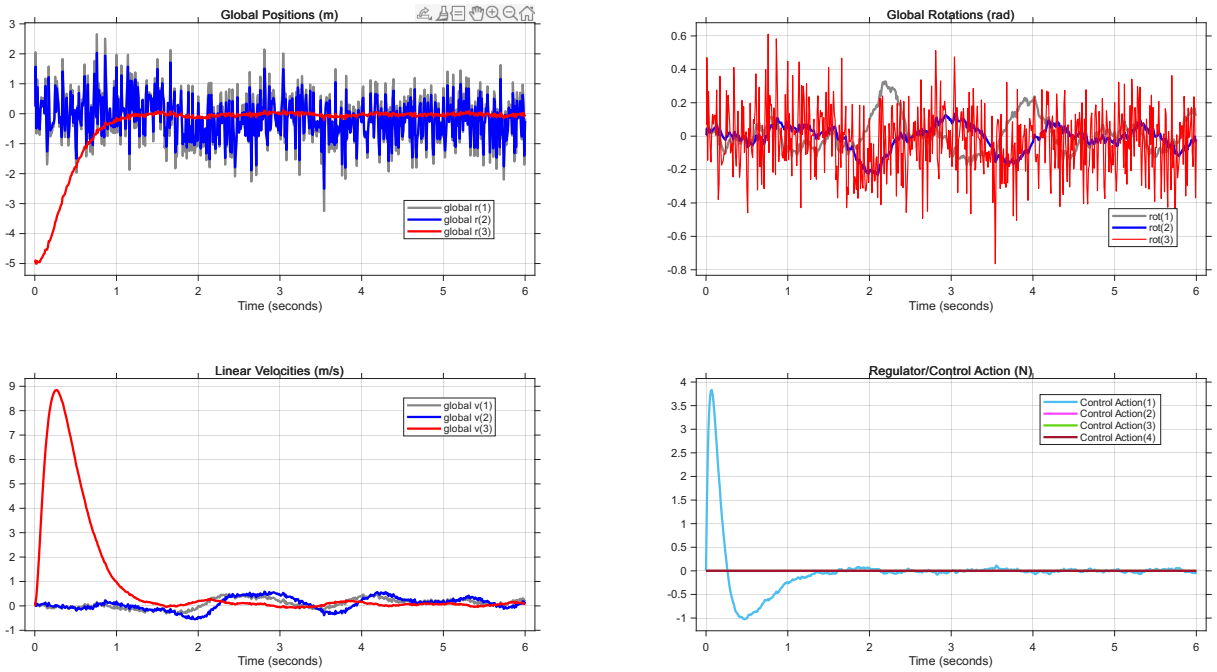


Figure 6: Q_0 **increased** with a factor of 7.4.

6.

While keeping R_0 at a fixed value, increasing Q_0 relative to the optimal setting causes the estimator to place greater trust in the measurements, resulting in noisier but more responsive state estimates. Conversely, decreasing Q_0 makes the estimator rely more heavily on the model, yielding smoother estimates that may lag behind true state changes, particularly during unmodeled dynamics or external disturbances. Physically, this means that the estimator trusts that the quadcopter's behavior follows the mathematical model more faithfully than it actually does, believing that the process noise (eg. wind gusts, as mentioned previously) have less influence in the dynamics than they have in reality.

Linear Model

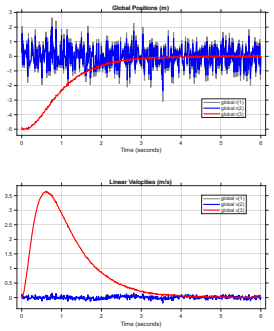


Figure 7: Q_0 **decreased** with a factor of 0.03.

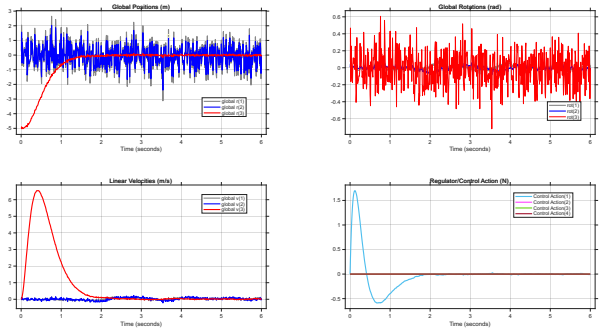


Figure 8: Q_0 **decreased** with a factor of 0.37.

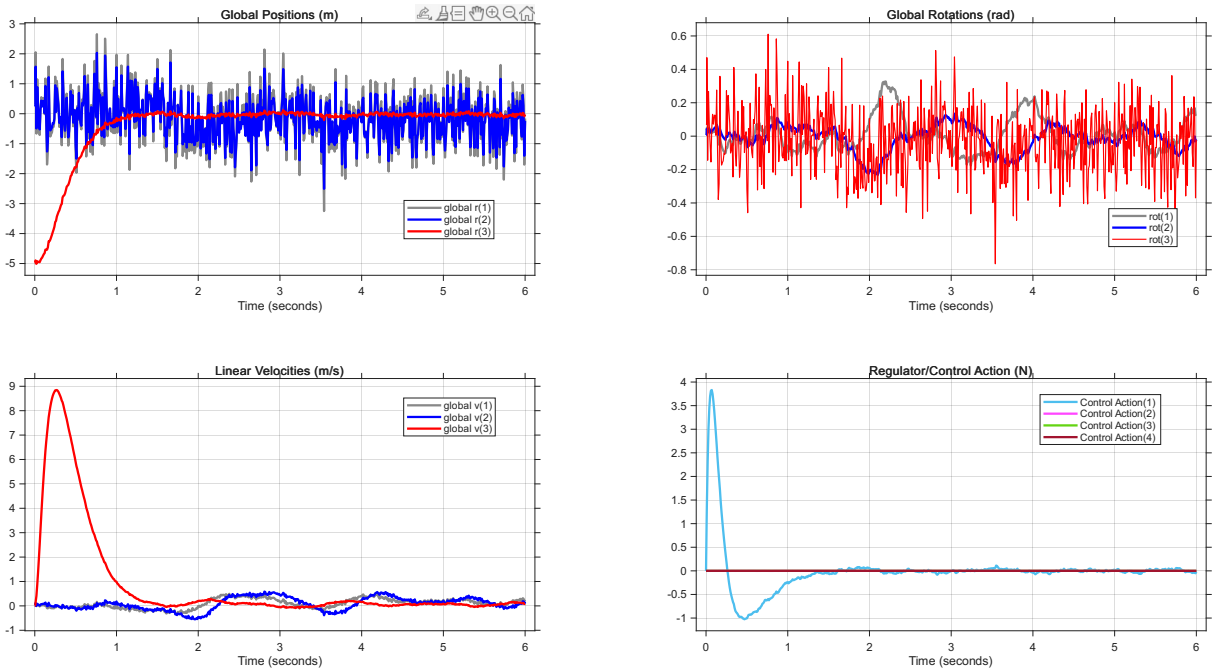


Figure 9: Q_0 **increased** with a factor of 7.4.

Non-linear Model

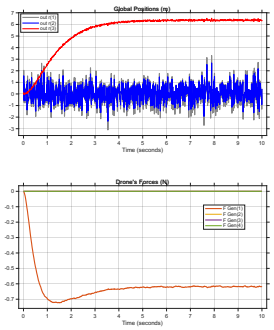


Figure 10: Q_0 **decreased** with a factor of 0.03.

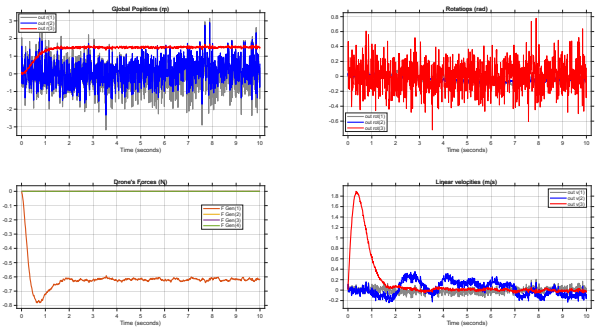


Figure 11: Q_0 **decreased** with a factor of 0.37.

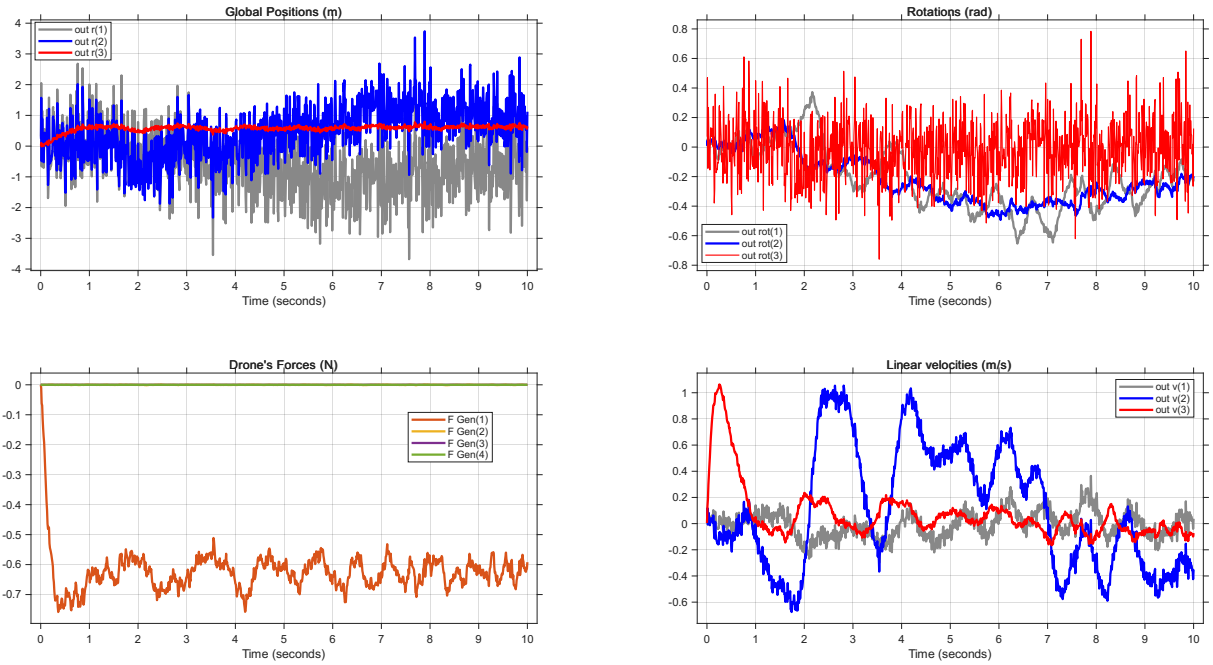


Figure 12: Q_0 **increased** with a factor of 7.4.

7.

Following much the same logic as for tuning Q_0, lowering R_0 below its true value causes the estimator to trust measurements excessively, making the state estimates more responsive to sensor noise and leading to erratic, high-frequency fluctuations in the estimated states. In other words, noise stems essentially from process noise. Raising R_0 above its true value has the opposite effect: the estimator relies more heavily on the model, producing smoother but more lagging estimates that respond slowly to actual changes in the quadcopter's motion.

Linear Model

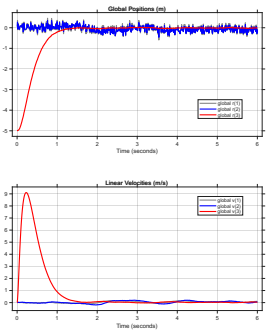


Figure 13: R_0 **decreased** with a factor of 0.03.

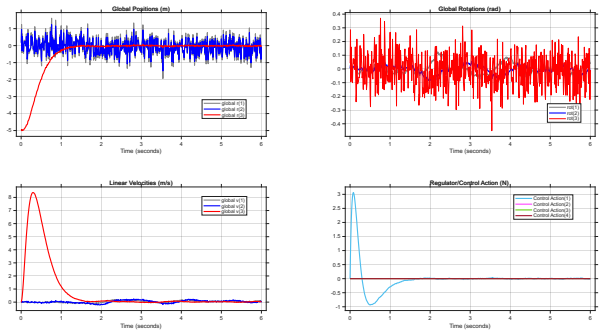


Figure 14: R_0 **decreased** with a factor of 0.37.

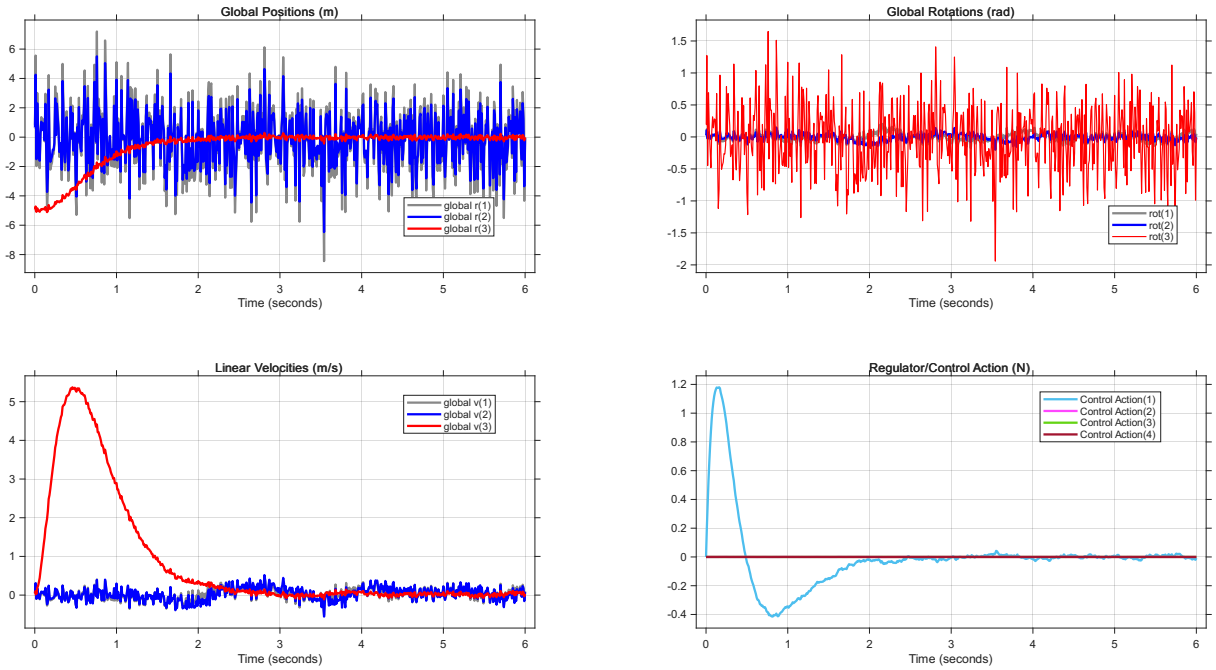


Figure 15: R_0 **increased** with a factor of 7.4.

Non-Linear Model

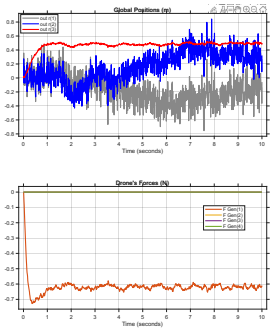


Figure 16: R_0 **decreased** with a factor of 0.03.

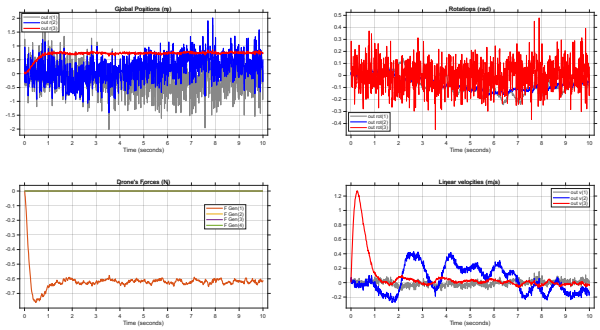


Figure 17: R_0 **decreased** with a factor of 0.37.

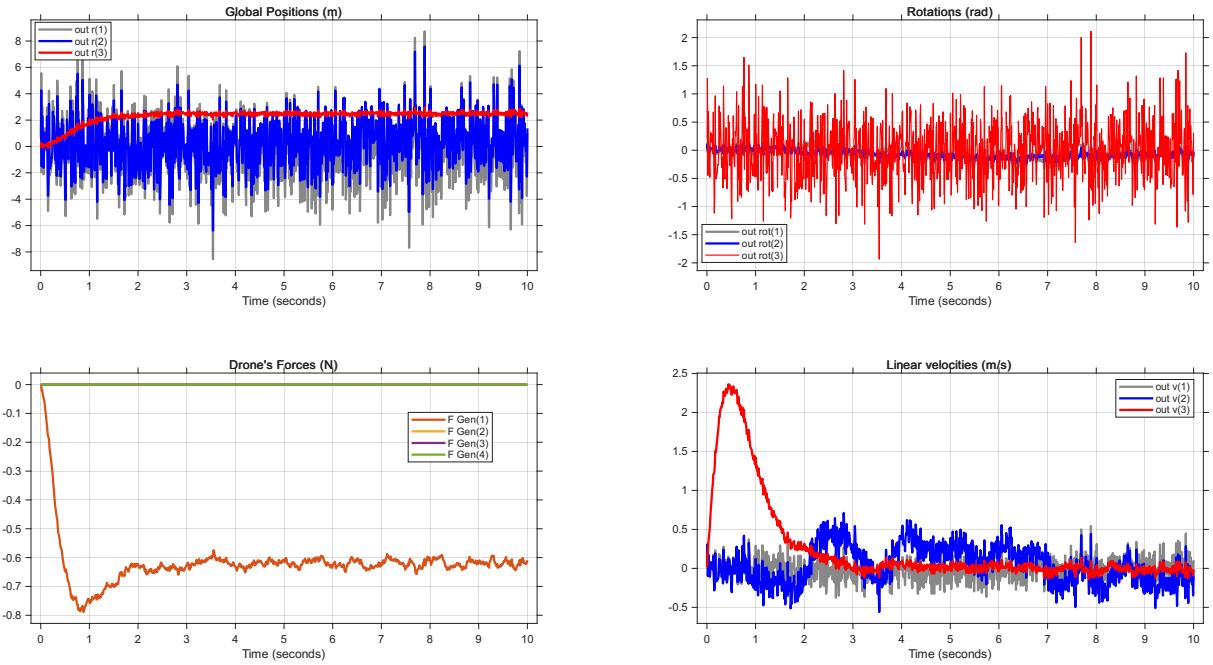


Figure 18: R_0 **increased** with a factor of 7.4.

8.

What was asked to do on this exercise was already answered in 6, **Non-linear Model** and 7, **Non-linear Model**.

2 LQR Servo Controller

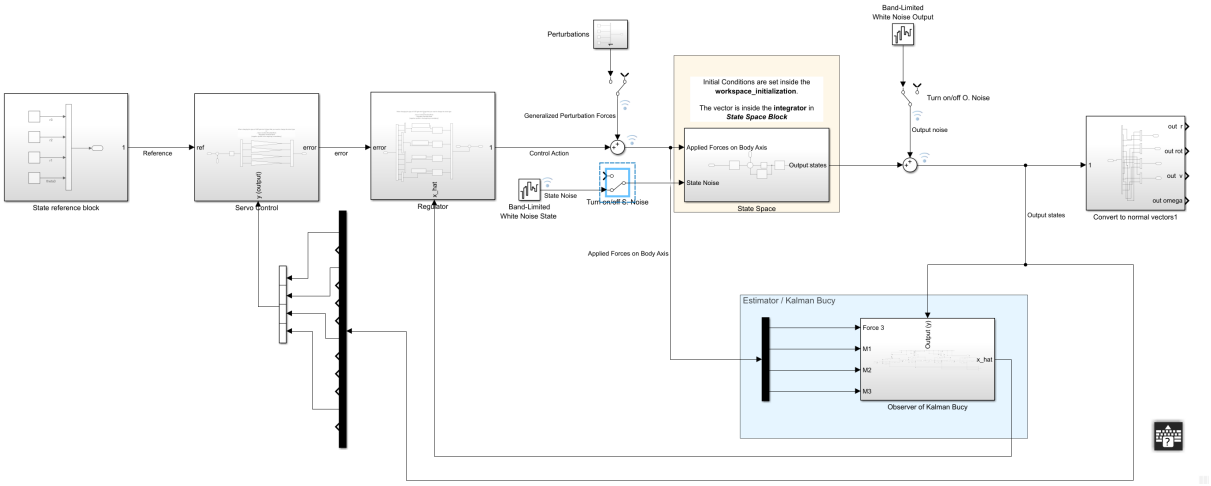


Figure 19: LQR – linear model Simulink.

9.

To design the LQR servo controller, the four decoupled subsystems were augmented with an integrator state x_i, defined as:

$$\dot{x}_i(t) = r(t) - y(t) = r(t) - Cx(t) \tag{5}$$

This yields the augmented open-loop system,

$$\begin{bmatrix} \dot{x}(t) \\ \dot{x}_i(t) \end{bmatrix} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} \begin{bmatrix} x(t) \\ x_i(t) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u(t) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} r(t) \tag{6}$$

and the corresponding control law,

$$u(t) = -\begin{bmatrix} K & k_i \end{bmatrix} \begin{bmatrix} x(t) \\ x_i(t) \end{bmatrix} \tag{7}$$

where K is the state feedback gain and k_i is the integral gain. Both were obtained simultaneously by solving a single LQR problem on the augmented system, minimizing the quadratic cost functional:

$$J = \int_0^\infty \left(\tilde{x}^T(t)\, Q\, \tilde{x}(t) + u^T(t)\, R\, u(t) \right) dt \tag{8}$$

where $\tilde{x} = \begin{bmatrix} x^T & x_i \end{bmatrix}^T$ is the augmented state vector.

This procedure was applied independently to each of the four decoupled subsystems of the linearized drone model (vertical (r_3), lateral (r_2), longitudinal (r_1), and yaw (θ_3)) yielding a separate state feedback gain vector K and integral gain k_i for each subsystem. For the remaining states, their control is done exclusively by the regulator.

In terms of the observer's role in the system, when working in conjunction with the LQR servo controller, initial values different than zero were set for the estimated states to prevent cases where due to the distance to the actual value of the state, in addition to noise and other perturbations, the estimates cannot converge. This caused the Simulink models to take an extremely long time to run, which became infeasible as the development of the controller progressed. Due to this, the initial value for every estimated state was set to zero.

10.

The matrices Q and R found in the cost functional (equation 8) are the two tuning parameters for arriving at an optimally controlled system. As seen in the equation, matrix Q is multiplied by the state vector, and thus is the parameter that controls the speed of the system's response, with matrix R, multiplied by the input, controlling how much actuator energy is expended in getting the system to converge to the reference. Optimal control is found by obtaining values for Q and R that result in a good compromise between performance and actuator energy, taking into account the limitations of the system, for example, in terms of available power and motor saturation limits.

In concrete terms, high values of Q penalize the deviation from the reference, making the system converge quicker, at the expense of higher energy being required for satisfying the control action and with a risk of saturation occurring. It should be noted how, in much the same way as R_0 and Q_0 in the Kalman-Bucy observer, only the relative difference in values between the two matrices is relevant. For example, if both are scaled by a constant α, the result is that it can be placed outside the cost functional integral, leaving the original integral scaled by α, leaving the optimal control path unchanged.

As separate servo controllers were implemented for each subsystem, the dimensions of the R and Q matrices are 3×3 or 5×5, not 4×4 or 2×2, since the system under consideration is formulated in its augmented form (equation 6) which includes an extra state variable pertaining to the integrator.

To tune the controller, weighting matrices Q and R were selected using Bryson's rule [1], whereby each diagonal entry is set as the inverse of the square of the maximum acceptable value for the corresponding state or input,

$$Q_{ii} = \frac{1}{x_{i,\max}^2}, \qquad R_{jj} = \frac{1}{u_{j,\max}^2} \tag{9}$$

The maximum acceptable state deviations were chosen based on physical constraints of the Parrot Mambo quadcopter platform, as well as general desired output requirements: a position error of 0.1 m, linear velocities of 0.1 m/s, roll and pitch angles of 0.2 rad (consistent with the small-angle linearization assumption), a yaw angle of $\pi/4$ rad, and angular velocities of 0.5 rad/s. The maximum acceptable inputs were derived from the motor saturation limit $u_{i,\max} = 500$ and the hover command $u_{\text{hover}} \approx 237$, yielding a maximum command deviation of $\Delta u \approx 263$. Applying the force and moment mappings of Equation (2.15), this corresponds to a maximum vertical force deviation of $F_{3,\max} \approx 2.2$ N, maximum roll and pitch moments of $M_{1,2,\max} \approx 0.002$ Nm, and a maximum yaw moment of $M_{3,\max} \approx 8.5 \times 10^{-4}$ Nm.

11.

The influence of the weighing matrices Q and R on the time response of the vertical movement servo controller was analyzed in the linear system by first feeding back only the output into the regulator, without using the state observer. This configuration of the Simulink system, which will also be used in the next section, is shown in figure 20.

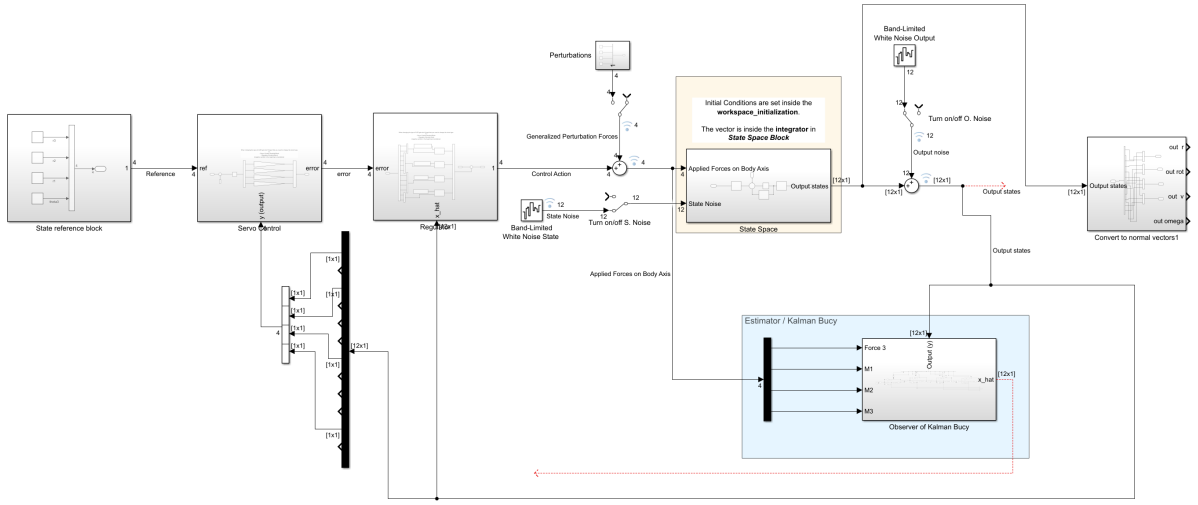


Figure 20: Linear system without Kalman-Bucy observer.

To aid the tuning, Bryson's rule was applied with four different sets of maximum allowable state and input values: $x_{\max} = 0.001, 0.1, 0.5$, and 10, while keeping the maximum input fixed at $F_{3,\max} \approx 2.2\,\text{N}$ in all cases. The maximum allowable state deviations were changed for all states except for the yaw angle, which was kept as $\frac{\pi}{4}$ in every case. This yields the following diagonal weighting matrices and scalar R values:

$$Q_1^{(0.001)} = \text{diag}(10^6,\ 10^6,\ 10^6), \quad R_1^{(0.001)} = 2.1095$$
$$Q_1^{(0.1)} = \text{diag}(100,\ 100,\ 100), \quad R_1^{(0.1)} = 2.1095$$
$$Q_1^{(0.5)} = \text{diag}(4,\ 4,\ 4), \quad R_1^{(0.5)} = 2.1095$$
$$Q_1^{(10)} = \text{diag}(0.01,\ 0.01,\ 0.01), \quad R_1^{(10)} = 2.1095$$

The corresponding LQR gains obtained for each case are:

$$K_1^{(0.001)} = [1792.6,\ 683.6], \quad k_{i,1}^{(0.001)} = -683.5$$
$$K_1^{(0.1)} = [11.937,\ 6.992], \quad k_{i,1}^{(0.1)} = -6.889$$
$$K_1^{(0.5)} = [2.446,\ 1.487], \quad k_{i,1}^{(0.5)} = -1.370$$
$$K_1^{(10)} = [0.163,\ 0.153], \quad k_{i,1}^{(10)} = -0.068$$

The simulations were carried out on the linearized vertical subsystem with a unit step reference $r_3^{\text{ref}} = 1\,\text{m}$, both without noise and with measurement and process noise (Figures 21, 22, 23, 24). The response shown in the figures are the value of the states and not the output, meaning that, for the case with noise, while the measurement noise is in the system, what is shown in the plots is the actual behavior of the drone (as per what the model simulates).

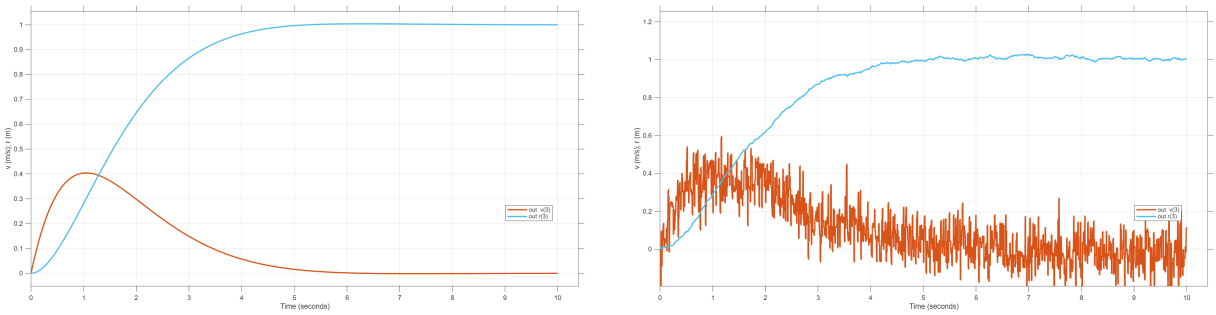


Figure 21: Linear system response to vertical unit step with no noise (left) and with noise (right), considering $Q = \text{diag}(10^6)$.

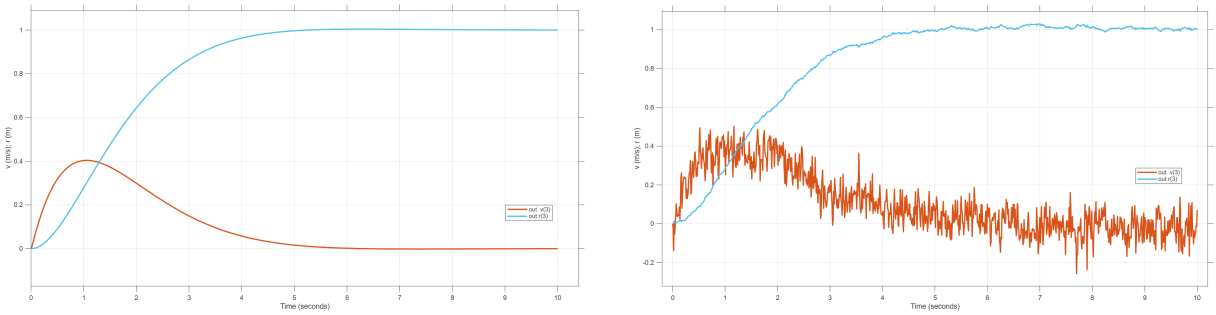


Figure 22: Linear system response to vertical unit step with no noise (left) and with noise (right), considering $Q = \text{diag}(100)$.

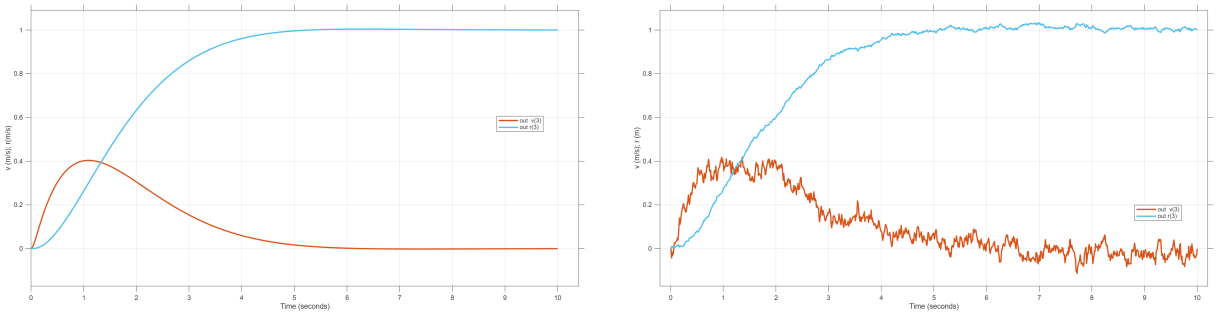


Figure 23: Linear system response to vertical unit step with no noise (left) and with noise (right), considering $Q = \text{diag}(4)$.

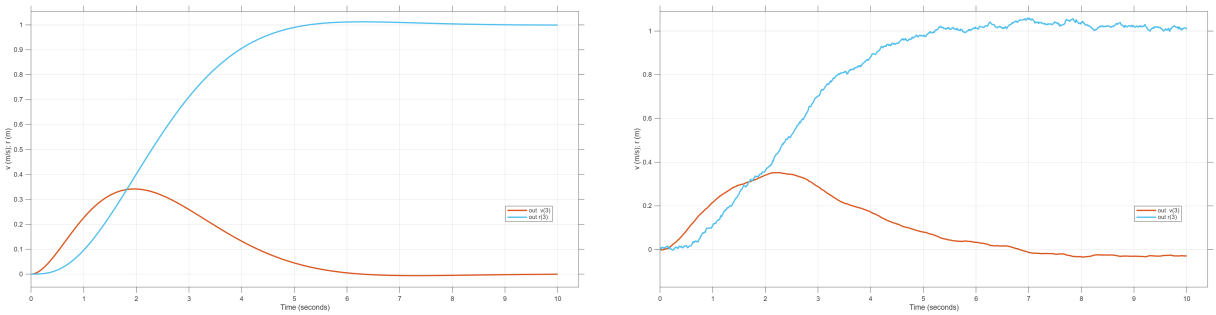


Figure 24: Linear system response to vertical unit step with no noise (left) and with noise (right), considering $Q = \text{diag}(0.01)$.

In the noise-free case, the response is remarkably similar across all four tunings. In all cases, r_3 converges smoothly to the reference with no overshoot, settling around $t \approx 5\,\text{s}$. The vertical velocity v_3 peaks at approximately $0.4\,\text{m/s}$ near $t \approx 1\,\text{s}$ for the three higher-Q cases, with the lowest-Q case showing a marginally lower peak of approximately $0.34\,\text{m/s}$ and a slightly slower settling time. Despite the gains differing by several orders of magnitude across the four cases, the visible effect on the time response is minimal.

When noise is introduced, all four cases exhibit similar behavior in terms of position response: r_3 tracks the reference showing a small noise amplitude, especially when compared with, v_3, which is heavily corrupted by noise throughout the simulation. Despite this, the velocity response shows a considerable decrease in noise as the value of Q is lowered, with the velocity response for $x_{max} = 10$ m showing deviations of similar magnitude to the position response. This can be attributed to the fact that when Q is higher, the system is constantly trying to correct every deviation introduced by the noise, leading to a jittery velocity response.

This near-insensitivity to the choice of Q and R can be attributed to the structure of the vertical subsystem. Since R remains constant across all cases, varying Q only rescales the relative weight between state penalties and input cost. However, because the vertical subsystem is a simple double integrator driven directly by F_3', with no coupling to other states, the LQR problem has limited freedom to produce qualitatively different closed-loop dynamics, with all tunings tested resulting in a stable, overdamped response that satisfies the reference tracking requirement, with little difference between them. The integral action further ensures zero steady-state error regardless of the gain magnitude, masking potential differences that might otherwise be visible.

12.

Following the same method as in the previous question, the servo controller for the second subsystem was tested without the observer for different values of Q, this time subjected to a unit step in the longitudinal direction. In this case, Q is a 5×5 diagonal matrix with the values in the diagonal being equal to the ones in the previous section, as shown below:

$$Q_2^{(0.001)} = \text{diag}(10^6,\ 10^6,\ 10^6,\ 10^6,\ 10^6), \quad R_2^{(0.001)} = 2.1095$$
$$Q_2^{(0.1)} = \text{diag}(100,\ 100,\ 100,\ 100,\ 100), \quad R_2^{(0.1)} = 2.1095$$
$$Q_2^{(0.5)} = \text{diag}(4,\ 4,\ 4,\ 4,\ 4), \quad R_2^{(0.5)} = 2.1095$$
$$Q_2^{(10)} = \text{diag}(0.01,\ 0.01,\ 0.01,\ 0.01,\ 0.01), \quad R_2^{(10)} = 2.1095$$

The corresponding LQR gains obtained for each case are:

$$K_2^{(0.001)} = [33.139,\ 93.490,\ 28.554,\ 15.190], \quad k_{i,2}^{(0.001)} = -15.190$$
$$K_2^{(0.1)} = [0.331,\ 0.936,\ 0.286,\ 0.152], \quad k_{i,2}^{(0.1)} = -0.152$$
$$K_2^{(0.5)} = [0.066,\ 0.188,\ 0.057,\ 0.031], \quad k_{i,2}^{(0.5)} = -0.030$$
$$K_2^{(10)} = [3.369 \times 10^{-3},\ 1.039 \times 10^{-2},\ 2.978 \times 10^{-3},\ 1.875 \times 10^{-3}], \quad k_{i,2}^{(10)} = -1.519 \times 10^{-3}$$

The results for each case are shown in figures 25, 26, 27, and 28.

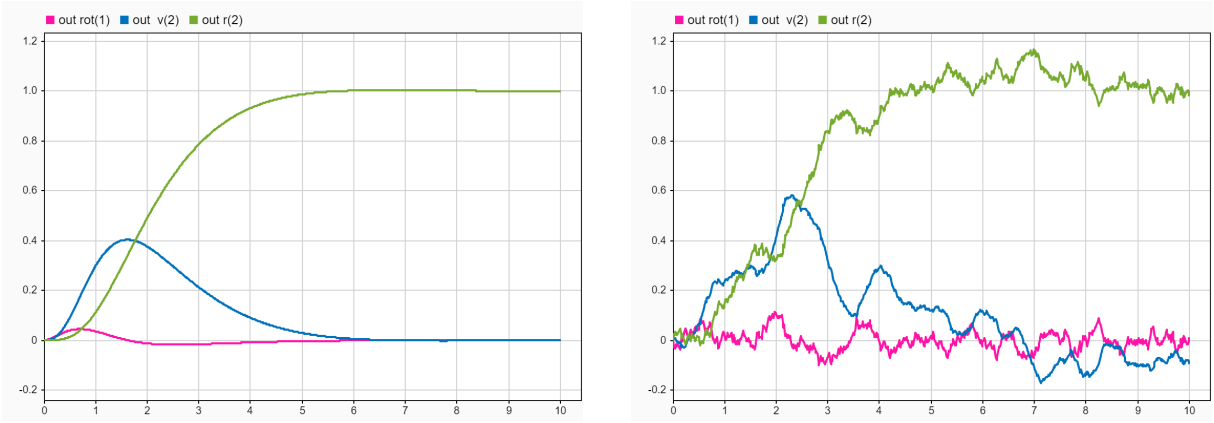


Figure 25: Linear system response to longitudinal unit step with no noise (left) and with noise (right), considering $Q = \text{diag}(10^6)$.

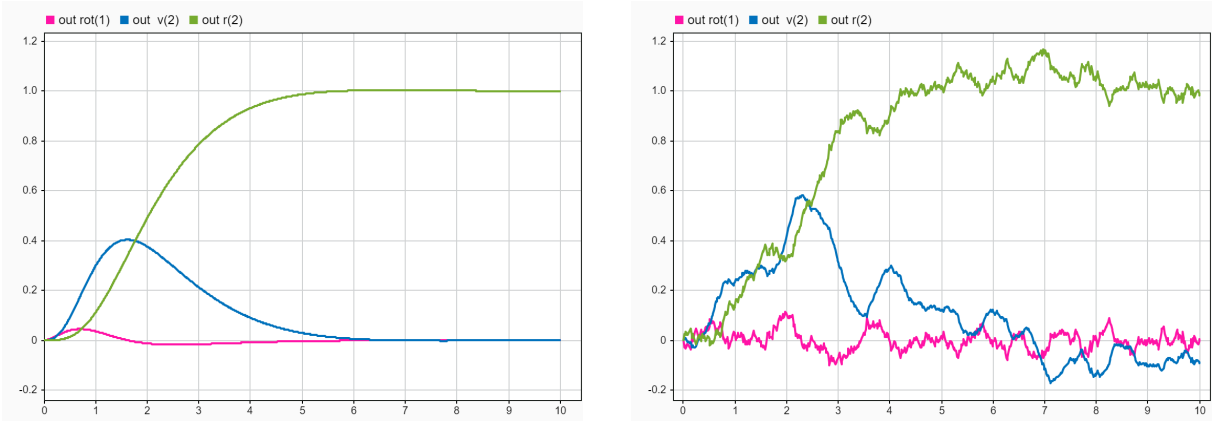


Figure 26: Linear system response to longitudinal unit step with no noise (left) and with noise (right), considering $Q = \text{diag}(100)$.

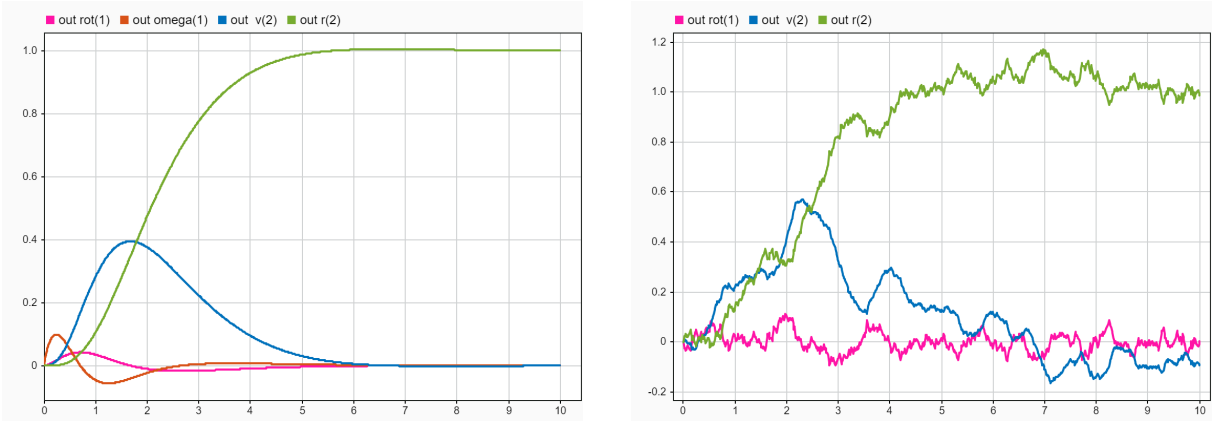


Figure 27: Linear system response to longitudinal unit step with no noise (left) and with noise (right), considering $Q = \text{diag}(4)$.

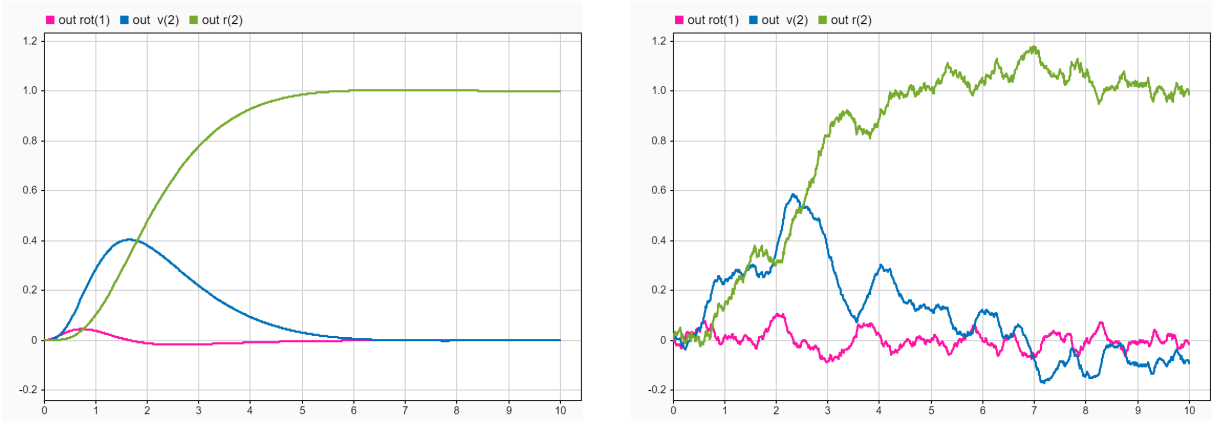


Figure 28: Linear system response to longitudinal unit step with no noise (left) and with noise (right), considering $Q = \text{diag}(0.01)$.

Much like in the previous section, for the case with no noise, different values of Q result in a very similar system response, due to many of the same factors explained previously. The main difference when compared to the vertical subsystem is found when analyzing the results that include process and sensor noise. Unlike before, for the longitudinal movement, there is no appreciable difference in the way the system reacts to the noise, with its amplitude being very similar between different values of Q. It should be noted that while consistently present, the plots show that the noise is much less intrusive than it was for the vertical subsystem (in the tests for the highest value of Q)

Before moving into the next section, where the Kalman-Bucy observer is implemented, it should be noted that for the results previously shown, where the noisy states go into the regulator, the angular velocity $\dot{\theta}_2$ is completely controlled by the perturbations, oscillating at an extremely high frequency, with an amplitude of upwards of 5 rad/s. This is an unacceptable response which will be mostly eliminated by the observer.

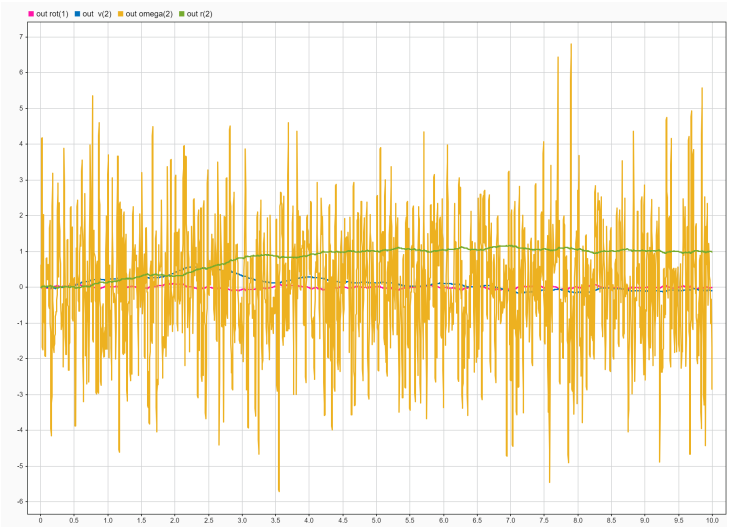


Figure 29: Noise on the states for a unit step in the longitudinal direction. Note the high frequency oscillation of $\dot{\theta}_2$.

13.

In this section, the states estimated by the Kalman-Bucy observer were fed into the regulator. A similar analysis to the one conducted previously was done, using the Simulink system shown in figure 30.

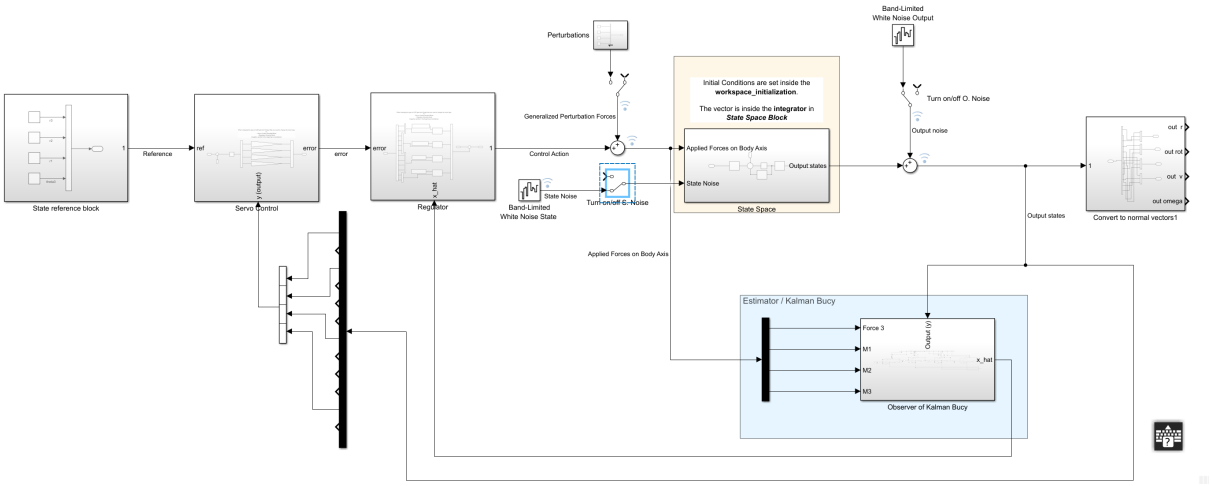


Figure 30: Linear system with LQR servo controller.

The results shown in figures 31 and 32 are for unit steps in the vertical and longitudinal directions, respectively. For comparison with the system without an observer, the results in this section are for the following Q and R values:

$$Q_1 = \begin{bmatrix} 4 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 4 \end{bmatrix} \qquad Q_2 = \begin{bmatrix} 4 & 0 & 0 & 0 & 0 \\ 0 & 25 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 & 0 \\ 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 4 \end{bmatrix}$$

$$R_1 = 2.1095 \qquad R_2 = 4.3342 \times 10^3$$

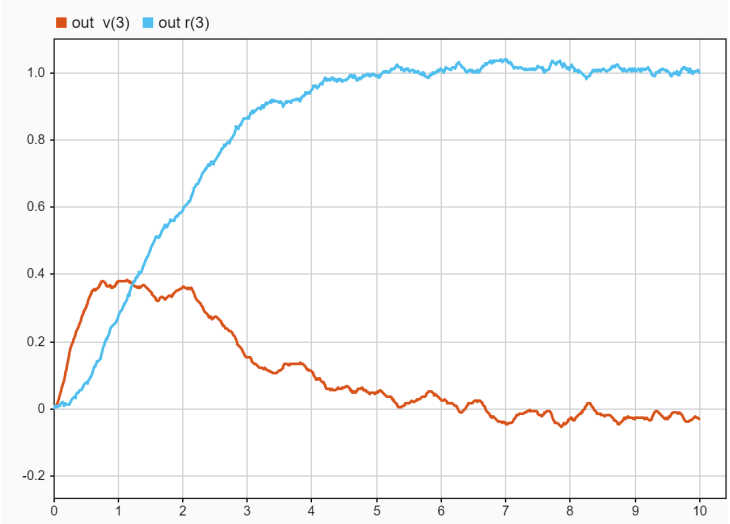


Figure 31: Response of the states for a unit step in the vertical direction.

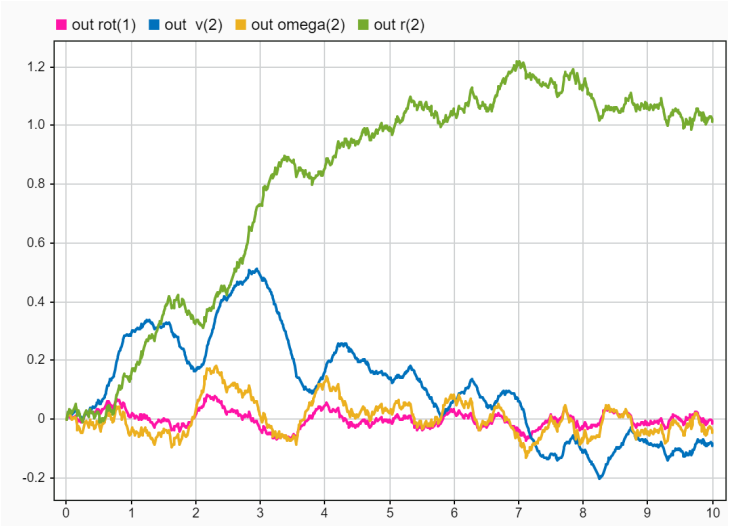


Figure 32: Response of the states for a unit step in the longitudinal direction.

Continuing the note on the noise of the angular velocity mentioned in the end of the previous section, the observer feedback dramatically reduces the noise seen in the behavior of each state, being especially noticeable for $\dot{\theta}_2$. Figure 33 is the same as figure 32 but with the same vertical axis scale as 29 from section 12, for comparison.

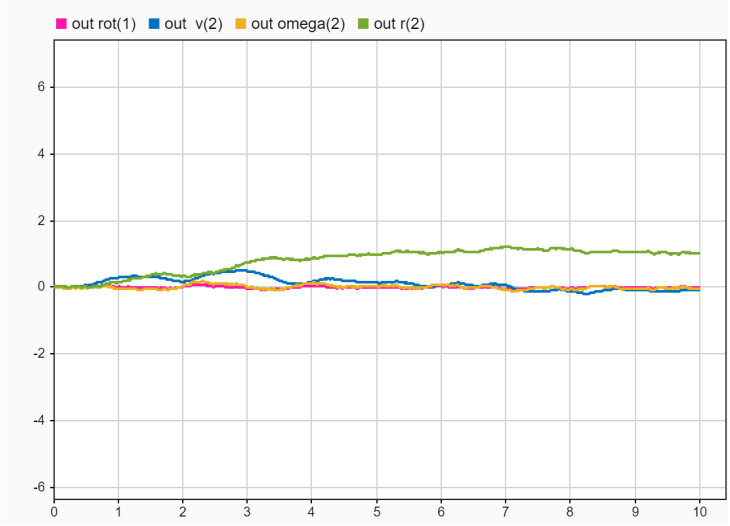


Figure 33: Response of the states for a unit step in the longitudinal direction, for comparing with figure 29.

14.

When the regulator and the state estimator were applied to the nonlinear model of the drone, using the same weighting matrices Q and R as in Section *13*, a rapid verification was conducted to assess whether the drone motors exhibited saturation under a vertical reference input.

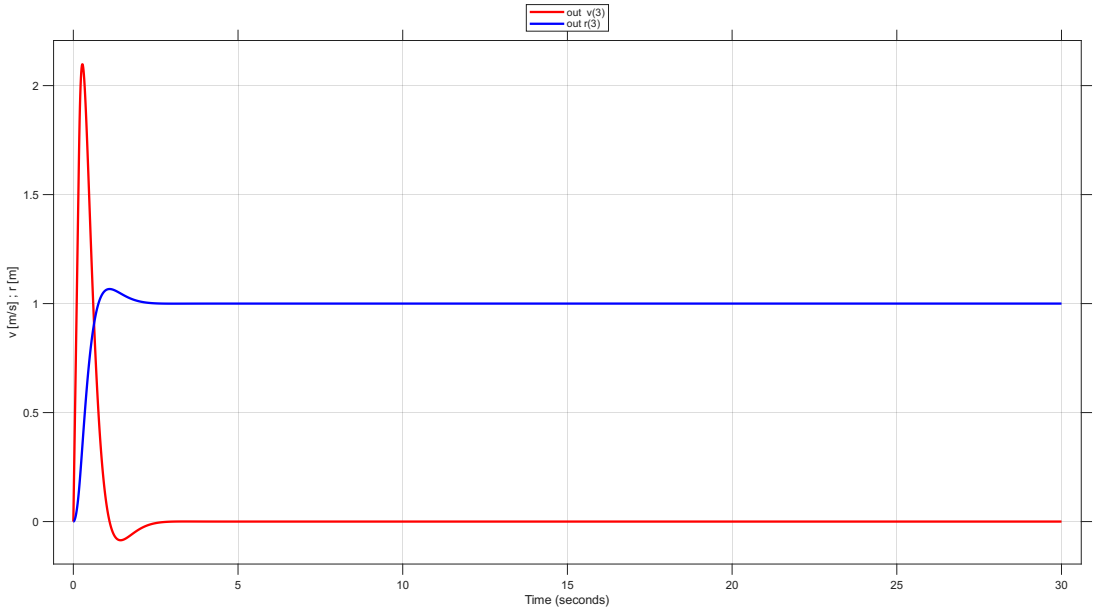


Figure 34: Response of the states for a unit step in the vertical direction for the non-linear system

With the reference applied above ($r_3 = 1$ m, i.e. go down 1 meter) there was no saturation applied:

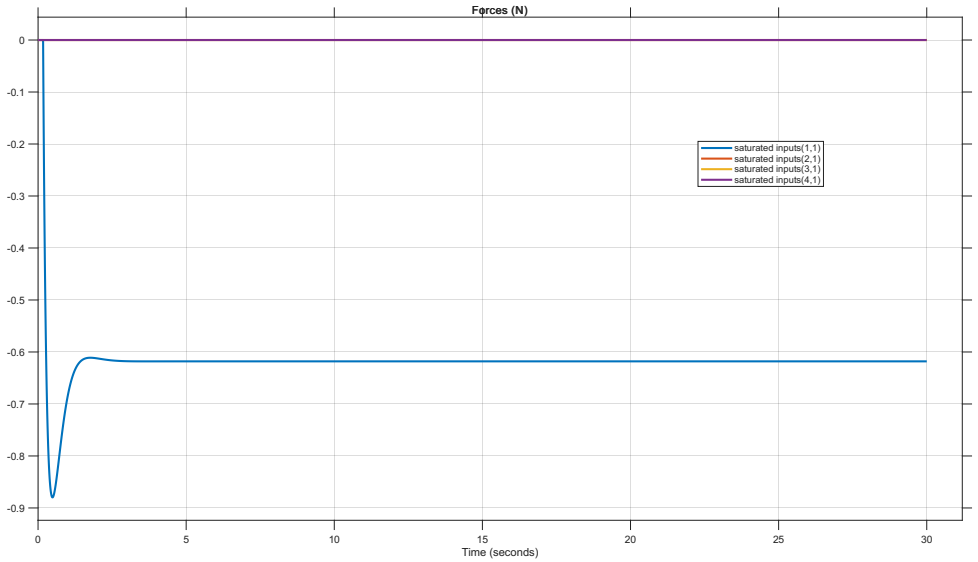


Figure 35: No saturation applied in the Forces.

Now, applying a reference of -15 in r_3 (go up 15 meters):

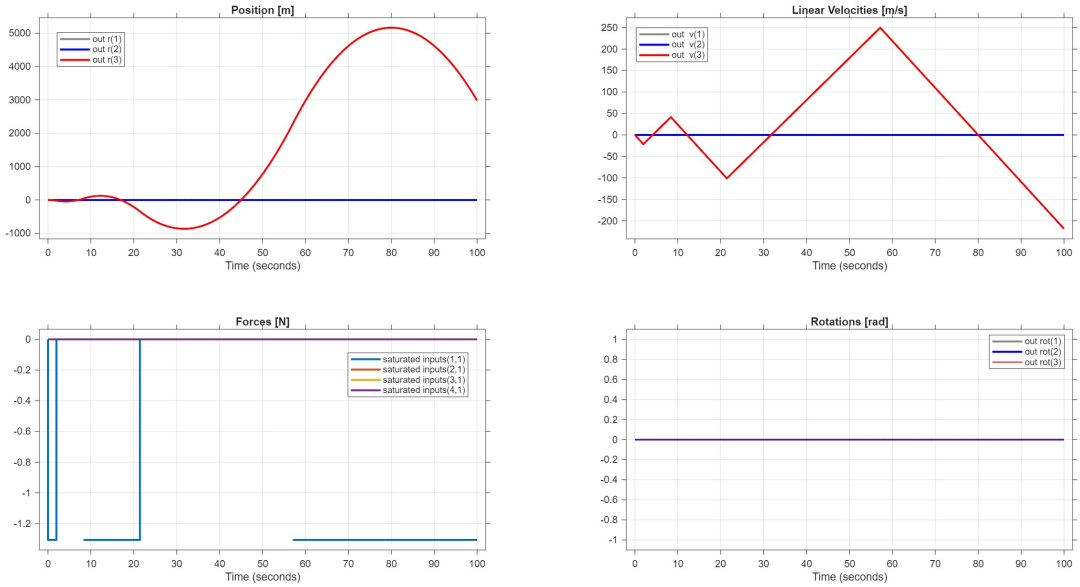


Figure 36: Saturation applied in the Forces. Vertical Force was saturated.

Since saturation occurs when the reference changes too quickly (for example, a high magnitude step or a ramp with too high of a slope, multiple requests of references), it is needed to update Q_0 and R_0 in order to adjust the state observer gain, L_k, allowing the state observer to better accommodate aggressive reference variations and mitigate the risk of saturation.

15.

Compared to previous position plots, the r_1 response to a step after having the drone rotated through θ_3 does not seem to follow a simple (linear system-like) curve, making it look like there is some sort of error associated to the feedback loops unaccounted for, even when there is no noise applied to the states or measurements (check question 16).

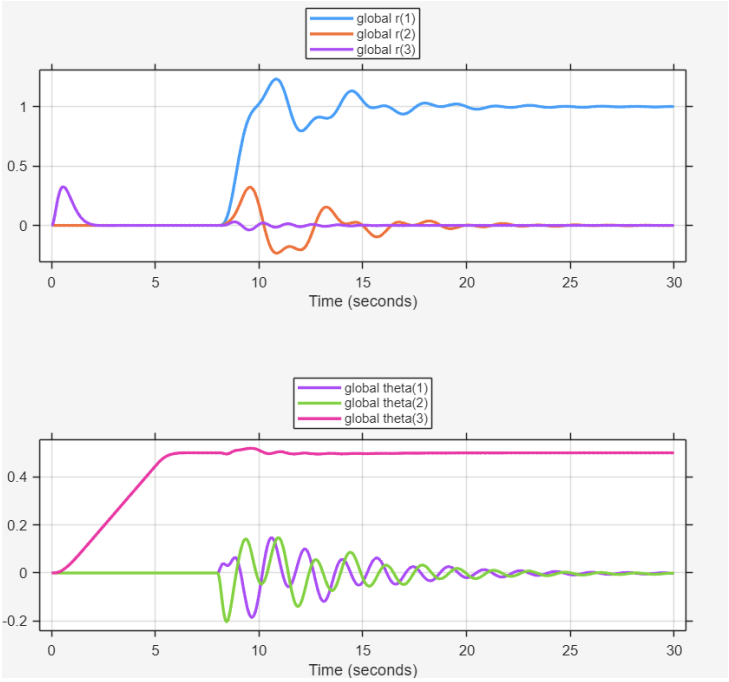


Figure 37: Global r and θ plots of a reference of a step $r_1 = 1$, having a $\theta_3 \neq 0$

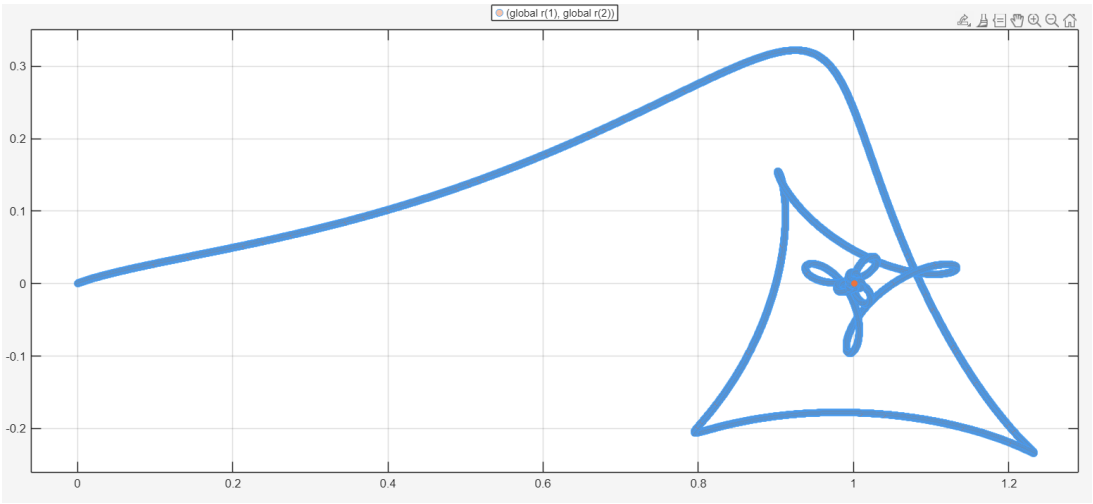


Figure 38: XY-plane trajectory of a reference of a step $r_1 = 1$, having a $\theta_3 \neq 0$

16. and 17.

With the updated reference frames from the position tracking error and the innovation term, the system (studied without noise) can now follow the global references more accurately, leading to a more stable behavior. Previously setting an r_1 in the reference effectively made the drone move in body-X's reference frame, since the controller did not have logic to "confirm" that both XY references were aligned. This makes the feedback loop do more work. This is confirmed when comparing both XY-plane trajectories, the first making paths that do not seem to have a purpose, while the second effectively converges at each time step.

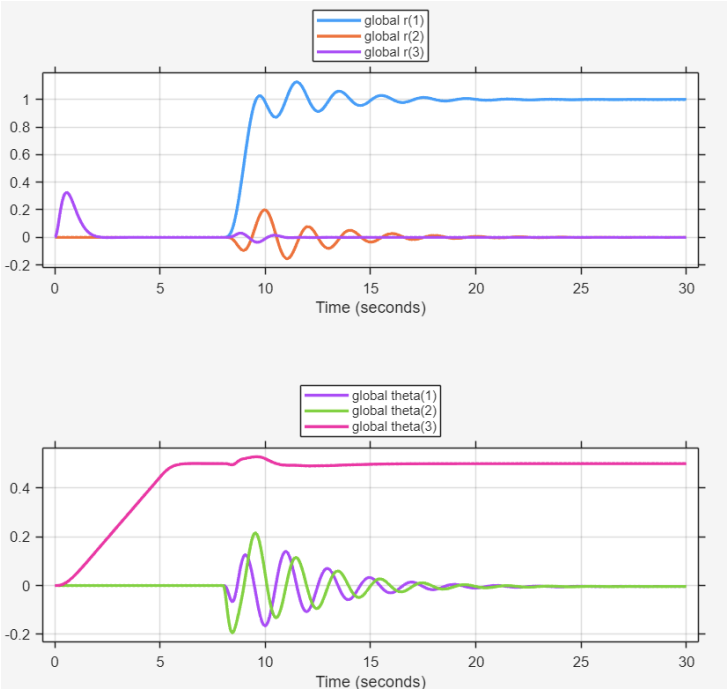


Figure 39: Global r and θ plots of a reference of a step $r_1 = 1$, having a $\theta_3 \neq 0$

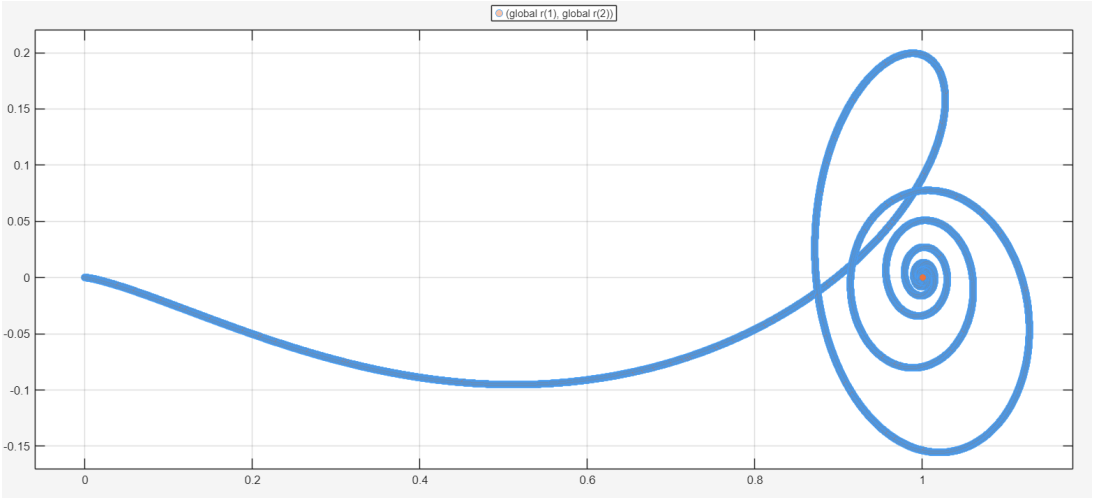


Figure 40: XY-plane trajectory of a reference of a step $r_1 = 1$, having a $\theta_3 \neq 0$

References

[1] *lqr - Linear-Quadratic Regulator (LQR) design - MATLAB*. URL: https://www.mathworks.com/help/control/ref/lti.lqr.html.

[2] Jorge Martins, Rui Coelho, and André Carvalho. *Rigid Body Dynamics: Modeling and Simulation of a Quadcopter. Optimal Control Course Notes*. Instituto Superior Técnico, University of Lisbon. 2022. URL: https://fenix.tecnico.ulisboa.pt/downloadFile/1407993358945355/quadcopterDynamics.pdf.



Optimal Control

Third Delivery

Students:

Gonçalo Tomé
Gustavo Rosa
Miguel Martins
Miguel Pimentel

June 2026

Contents

1 Discrete-Time Controller and Observer **3**
1.1 Discretizing the LQR Servo Controller . . . 3
1.1.1 Validation . . . 3
1.2 Kalman Filter (Observer) . . . 6
1.3 Validation . . . 6

2 Implementation on physical drone **10**
2.1 Real-life control gains . . . 10
2.2 Basic tasks . . . 10
2.2.1 Hovering . . . 10
2.2.2 Hovering, rotating in yaw and lateral movement . . . 12
2.3 Trajectory planning . . . 13

Glossary

Reference frames and states of the quadcopter model

Symbol	Name	Description
$\{O_I, X_I Y_I Z_I\}$	Inertial reference frame	Earth-fixed.
$\{O_n, X_n Y_n Z_n\}$	Quadcopter reference frame	Origin at the center of mass, principal axes.
R_n	Body-to-inertial rotation matrix	Orientation of the quadcopter with respect to the inertial frame
$\mathbf{v}_n = [v_1, v_2, v_3]^\top$	Linear velocity in the body frame	Velocity of the CM in the body frame.
$\boldsymbol{\omega}_n = [\omega_1, \omega_2, \omega_3]^\top$	Angular velocity	Quadcopter angular velocity in the body frame.
$\dot{\mathbf{v}}_n$	Linear acceleration	Derivative of $\mathbf{v}_n$.
$\dot{\boldsymbol{\omega}}_n$	Angular acceleration	Derivative of $\boldsymbol{\omega}_n$.
$\mathbf{V}_n = [\mathbf{v}_n^\top \ \boldsymbol{\omega}_n^\top]^\top$	Generalized velocity	Stacked linear and angular velocities.
$\dot{\mathbf{V}}_n$	Generalized acceleration	Used in $\dot{\mathbf{V}}_n = M_r^{-1}(\cdots)$.
$\mathbf{r}_n = [r_1, r_2, r_3]^\top$	Quadcopter position	Position of the CM in the inertial frame.
$\dot{\mathbf{r}}_n$	Velocity of the position	$\dot{\mathbf{r}}_n = R_n \mathbf{v}_n$.

Physical and inertial parameters

m_n	Quadcopter mass	Total mass.
$\mathbf{J}_n = \mathrm{diag}(J_{11}, J_{22}, J_{33})$	Inertia tensor	In the basis of the body's principal axes.
J_{11}, J_{22}, J_{33}	Principal moments of inertia	About X_n, Y_n, Z_n.
g	Gravitational acceleration	Scalar $\approx 9.81\ \mathrm{m/s^2}$.
$\mathbf{g}$	Gravity vector (inertial)	$[0, 0, -g]^\top$.
$\mathbf{g}_n$	Gravity in the body frame	$\mathbf{g}_n = R_n^\top \mathbf{g}$.

Forces, moments, and quadcopter inputs

T_i	Thrust of rotor i	Force along $+Z_n$, $T_i = K_T \Omega_i^2$.
τ_i	Reaction moment of rotor i	Torque about Z_n, $\tau_i = K_\tau \Omega_i^2$.
F_1, F_2, F_3	Body force components	Resultant of the forces from the 4 rotors in the body frame
M_1, M_2, M_3	External moment components	Resultant moments (roll, pitch, yaw).
L	Arm length	Distance from the center to the rotor.
K_T	Thrust constant	Relates Ω_i^2 to T_i.
K_τ	Torque constant	Relates Ω_i^2 to τ_i.
Ω_i	Angular velocity of rotor i	In rad/s.
u_i	Motor i command	Input signal (0 to $u_{i\,\max}$).
$K_{u\to\Omega^2}$	Input-to-Ω^2 gain	$\Omega_i^2 = K_{u\to\Omega^2} u_i$.
$u_{i\,\max}$	Command saturation	Maximum value of u_i.

Main matrices of the rigid body model

$\mathbf{M}_r$	Rigid mass matrix	$\begin{bmatrix} m_n I & 0 \\ 0 & \mathbf{J}_n \end{bmatrix}$.
$\mathbf{C}_{rn}(\boldsymbol{\omega}_n)$	Inertial nonlinear matrix	Mass/inertia parameters of Coriolis, centrifugal, and gyroscopic forces (moments).
$\mathbf{F}_n^{(\mathrm{gen})}$	Generalized forces	$[\mathbf{F}_n^\top \ \mathbf{M}_n^\top]^\top$.
$\mathbf{G}_{rn}$	Generalized gravitational forces	$[m_n \mathbf{g}_n^\top \ 0^\top]^\top$.

Symbols used in the model linearization

$\mathbf{v}_n', \boldsymbol{\omega}_n'$	Velocity variations	Small variations around equilibrium.
F_3', M_1', M_2', M_3'	Input variations	Deviations from the equilibrium point.
$\theta_1', \theta_2', \theta_3'$	Small angles	Linear roll, pitch, and yaw.
$\mathbf{x}$	Linearized state vector	Stacks $r_i, \dot{r}_i', \theta_i', \dot{\theta}_i'$.
$\mathbf{u}$	Linearized input vector	$[F_3' \ M_1' \ M_2' \ M_3']^\top$.
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	State matrices	Model $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$.
$H_1(s) \dots H_4(s)$	Transfer functions	Input-output relationships for each axis.

1 Discrete-Time Controller and Observer

To build a model that more closely relates what characteristics the control actions will have in the drone's hardware, the simulation controller must be converted from continuous-time to discrete-time. To discretize the system, both the reference input and the output states passed through a zero-order hold (ZOH, discretization method), outputting only the values of the previous "multiple of 5 ms" time sample (T_s).

In practice, discretization alters the observer's inputs (the output states), which typically introduces slight behavioral differences when tracking dynamic references such as ramp inputs. Furthermore, transitioning to the discrete-time domain requires updating the linearized system matrices A and B, while C remains unchanged. As a consequence, the equations used to calculate the LQR and servo gains must be modified to account for these discrete-time dynamics.

$$A_{Discrete} = e^{AT_s} \qquad B_{Discrete} = \int_0^{T_s} e^{A\tau} d\tau B \tag{1}$$

Note: In this report, $A_{Discrete}$ and $B_{Discrete}$, discretized matrices, will be referred to simply as A and B for example.

These matrices can be directly obtained using the `c2dm` function in MATLAB.

1.1 Discretizing the LQR Servo Controller

For the LQR Controller, the servo augmented state system should be applied to find its gains:

$$\hat{A} = \begin{bmatrix} A & 0 \\ -T_s C & I \end{bmatrix} \quad \hat{B} = \begin{bmatrix} B \\ 0 \end{bmatrix} \quad \hat{C} = \begin{bmatrix} C & 0 \end{bmatrix} \quad \hat{K} = -\begin{bmatrix} K & k_i \end{bmatrix} \tag{2}$$

Theoretically, the gains can be obtained through optimal state feedback control for each decoupled system. This involves solving the Algebraic Riccati Equation (4), where $Q \in \mathbb{R}^{n,+} \setminus \{0\}$ and $R \in \mathbb{R}^{+}$ represent the LQR weighting parameters:

$$K = (B^T P B + R)^{-1} B^T P A \tag{3}$$

$$P = A^T P A - A^T P B (B^T P B + R)^{-1} B^T P A + Q \tag{4}$$

In MATLAB, this approach can be applied by simply using $dlqr$ function from Control System Toolbox.

1.1.1 Validation

To validate the new controller, the states were directly fed back in the inner loop and the output measurements were fed back in the outer loop. The same was done on the continuous model. The values used to obtain the controller gains are using $R = 1$ with varying Q, are shown in table 1

Table 1: Q values used to obtain controller gains

Decoupling	Q
1. Vertical, r_3	$\begin{bmatrix}10 & 1 & 100\end{bmatrix}$
2. Lateral, r_2 and θ_1	$\begin{bmatrix}10 & 1 & 1 & 1 & 100\end{bmatrix}$
3. Frontal, r_1 and θ_2	$\begin{bmatrix}10 & 1 & 1 & 1 & 100\end{bmatrix}$
4. Yaw, θ_3	$\begin{bmatrix}10 & 1 & 100\end{bmatrix}$

The first example shows the response to a step of $r_3 = -1.1$ m, which is correctly tracked and yields the same behavior in both models.

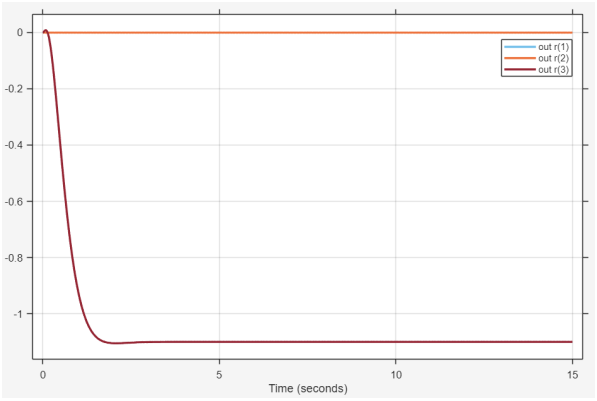


Figure 1: Continuous Model - Step on $r_3 = -1.1$ m

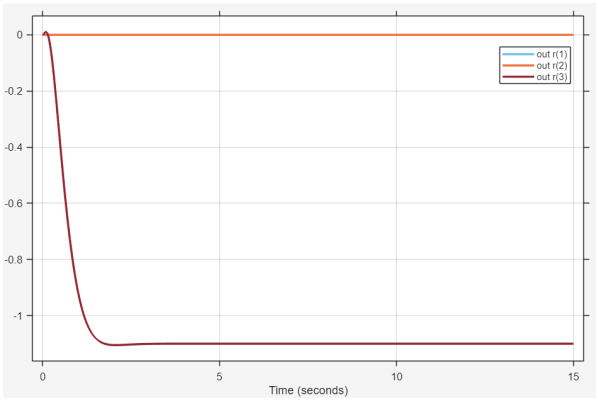


Figure 2: Discrete Model - Step on $r_3 = -1.1$ m

Keeping the same step on r_3, when $r_1 = 0.8$ m is included, the response also says the same between models in practice.

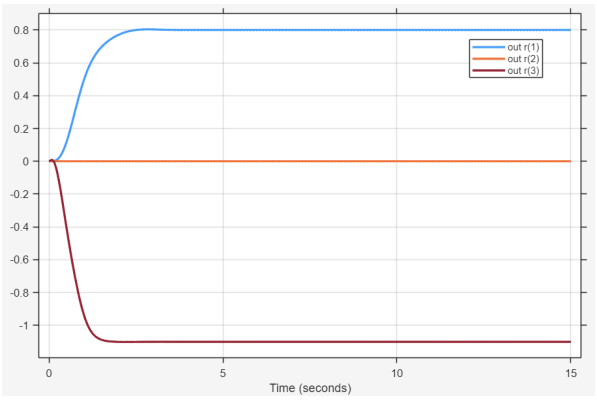


Figure 3: Continuous Model - Combination of step references on $r_1 = 0.8$ m and $r_3 = -1.1$ m - Position Plot

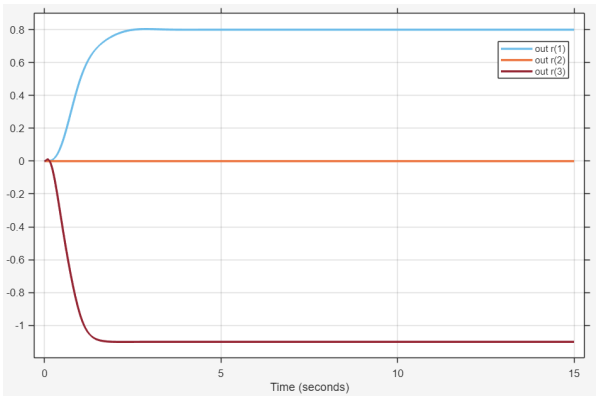


Figure 4: Discrete Model - Combination of step references on $r_1 = 0.8$ m and $r_3 = -1.1$ m - Position Plot

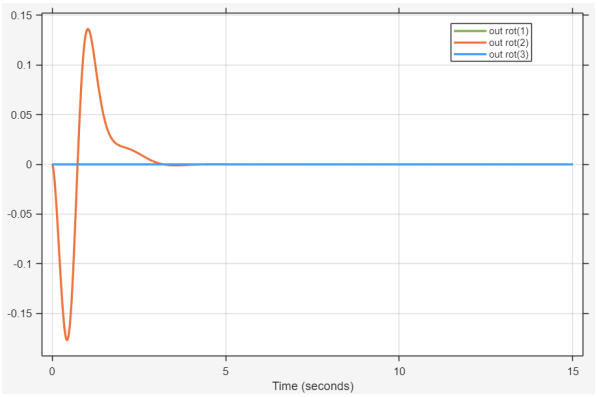


Figure 5: Continuous Model - Combination of step references on $r_1 = 0.8$ m and $r_3 = -1.1$ m - Rotation Plot

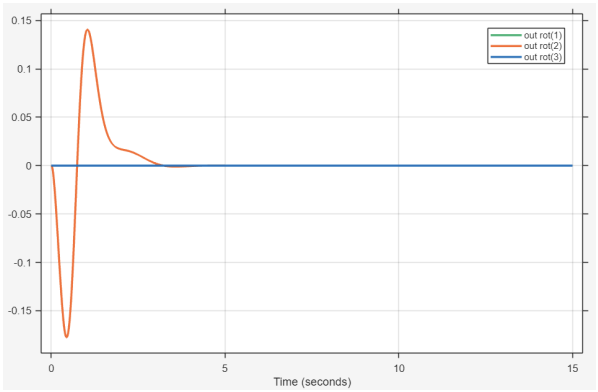


Figure 6: Discrete Model - Combination of step references on $r_1 = 0.8$ m and $r_3 = -1.1$ m - Rotation Plot

Finally, an initial rotation on $\theta_3 = 1$ rad is imposed before moving on r_1, which creates some problems for the drone's reference tracking initially.

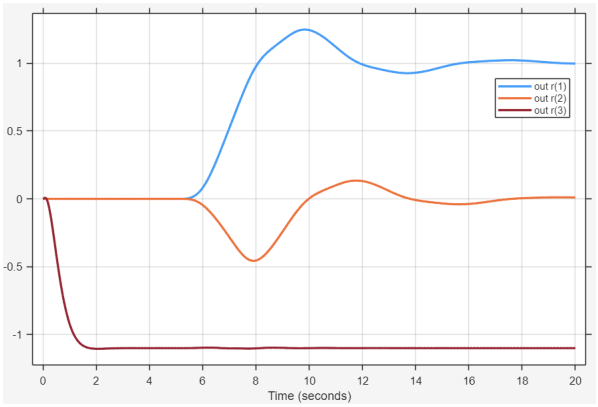


Figure 7: Continuous Model - Combination of a ramp on r_1 with a initial rotation on θ_3 - Position Plot

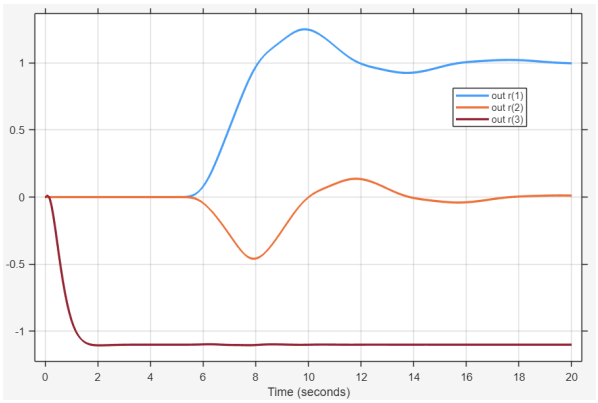


Figure 8: Discrete Model - Combination of a ramp on r_1 with a initial rotation on θ_3 - Position Plot

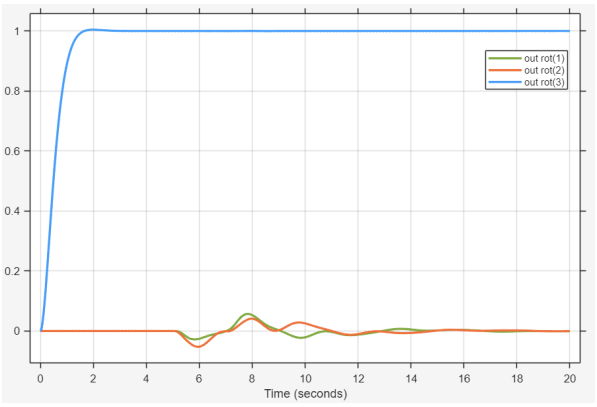


Figure 9: Continuous Model - Combination of a ramp on r_1 with a initial rotation on θ_3 - Rotation Plot

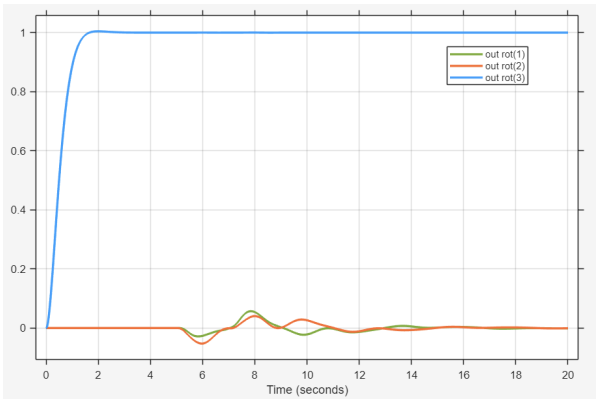


Figure 10: Discrete Model - Combination of a ramp on r_1 with a initial rotation on θ_3 - Rotation Plot

Concluding this validation, the continuous and discrete time LQR Servo controllers are indistinguishable when comparing their response to the same reference.

1.2 Kalman Filter (Observer)

The estimation problem solved using the Kalman Filter uses the recursive expressions 5:

$$\hat{x}_{k+1} = A\hat{x}_k + Bu_k \tag{5a}$$
$$P_{k+1} = AP_kA^T + Q_0 \tag{5b}$$
$$L_k = P_{k-1}C^T(CP_{k-1}C^T + R_0)^{-1} \tag{5c}$$
$$\hat{x}_k = \hat{x}_{k-1} + L_k(y_k - C\hat{x}_{k-1}) \tag{5d}$$
$$P_k = (I - L_kC)P_{k-1} \tag{5e}$$

where equations 5a and 5b correspond to the prediction stage, in which the expected state $\hat{x}_k$ and the covariance matrix P_k are updated with the linear model. Equations 5c, 5d and 5e correspond to the update stage, where the Kalman gain L_k calculated and $\hat{x}_k$ is updated with the sensor information on the innovation term $y_k - C\hat{x}_{k-1}$. The same occurs with P_k.

Both the discrete-time Kalman Filter (KF) and the continuous-time Kalman-Bucy Filter (KBF) operate fundamentally as optimal linear observers. Their core architecture relies on information fusion: dynamically mixing a priori information (the internal predictive linear system model) with a posteriori information (real-time noisy measurements) to minimize estimation error covariance. Although the same process and measurement noise covariance matrices, Q_0 and R_0, are used in both filters, they are incorporated differently due to the distinct continuous-time and discrete-time formulations.

Two clear differences of the observers are the fact the Kalman Gain L_k is obtained at each measurement update time step while the Kalman-Bucy Gain is constant, making the system converge in an asymptotic manner to $\hat{x}$ and not being able to adapt the gain dynamically. There exists the finite time Kalman-Bucy Gain but this must be calculated offline with a given trajectory, which is impractical for many applications where the desired trajectory is not known a priori. The other difference is the non-existence of the covariance matrix P_k in the Kalman-Bucy Observer, which can be a useful metric to analyze, even-though it is not done in this report.

1.3 Validation

After building the whole control system (LQR Servo controller and Kalman Filter) some validation similar to section 1.1.1 while using the Q_0 and R_0 values from Delivery 2 and only turning on the output noise due to simulation time restrictions. The values for the controller gains were obtained the same way as previously (see table 1).

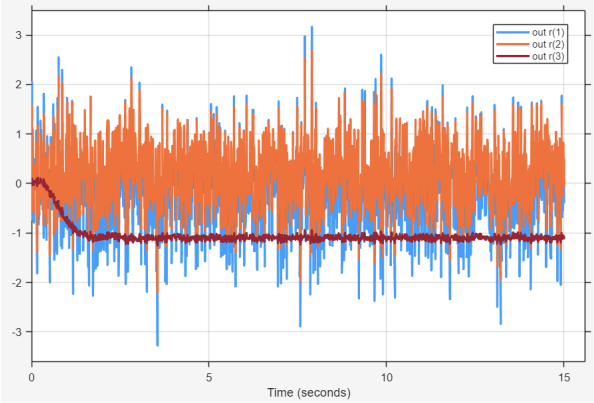


Figure 11: Continuous Model with noise - Step on $r_3 = -1.1$ m - Position Plot

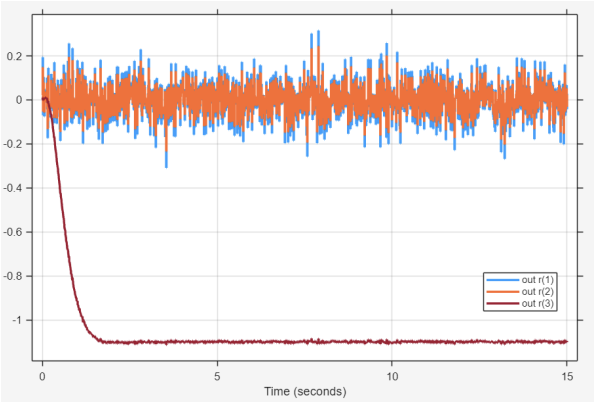


Figure 12: Discrete Model with noise - Step on $r_3 = -1.1$ m - Position Plot

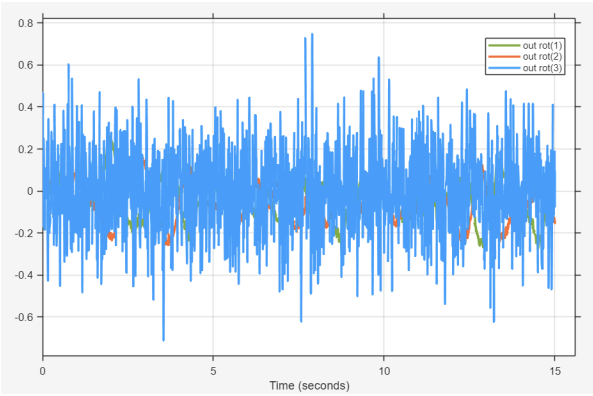


Figure 13: Continuous Model with noise - Step on $r_3 = -1.1$ m - Rotation Plot

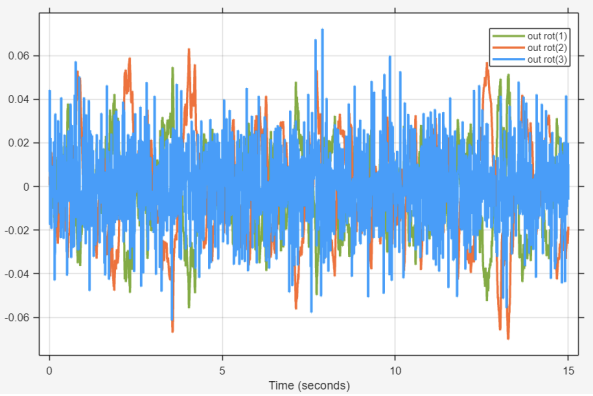


Figure 14: Discrete Model with noise - Step on $r_3 = -1.1$ m - Rotation Plot

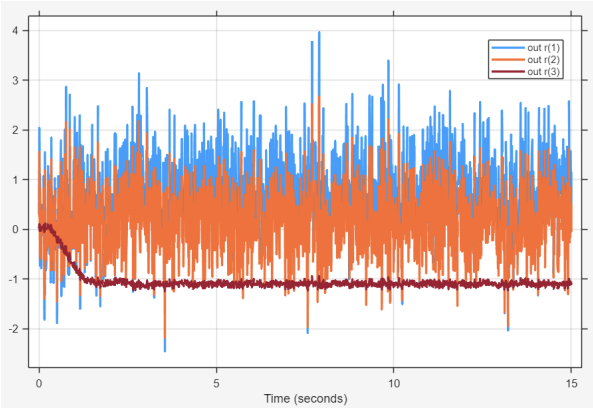


Figure 15: Continuous Model with noise - Combination of step references on $r_1 = 0.8$ m and $r_3 = -1.1$ m - Position Plot

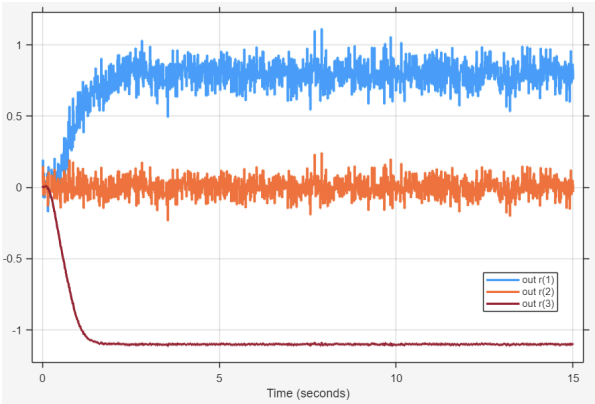


Figure 16: Discrete Model with noise - Combination of step references on $r_1 = 0.8$ m and $r_3 = -1.1$ m - Position Plot

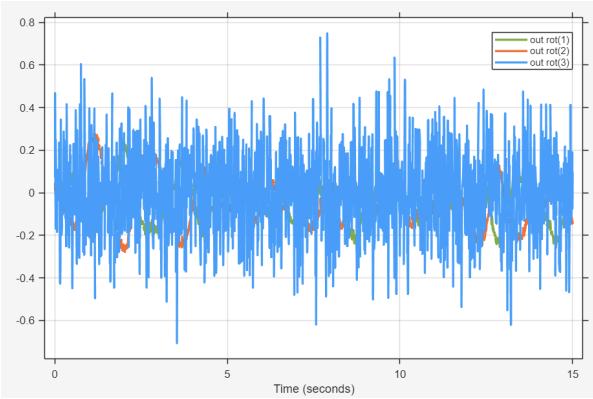


Figure 17: Continuous Model with noise - Combination of step references on $r_1 = 0.8$ m and $r_3 = -1.1$ m - Rotation Plot

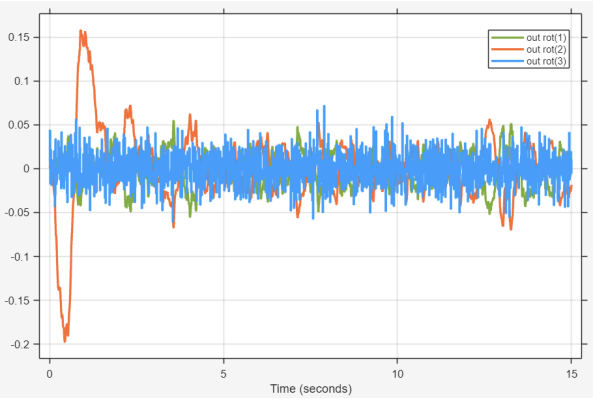


Figure 18: Discrete Model with noise - Combination of step references on $r_1 = 0.8$ m and $r_3 = -1.1$ m - Rotation Plot

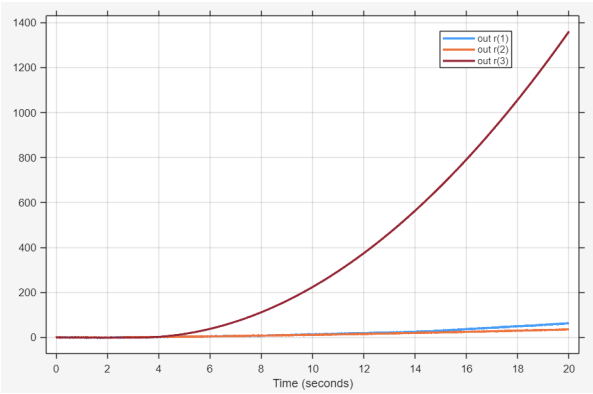


Figure 19: Continuous Model with noise - Combination of a ramp on r_1 with a initial rotation on θ_3 - Position Plot

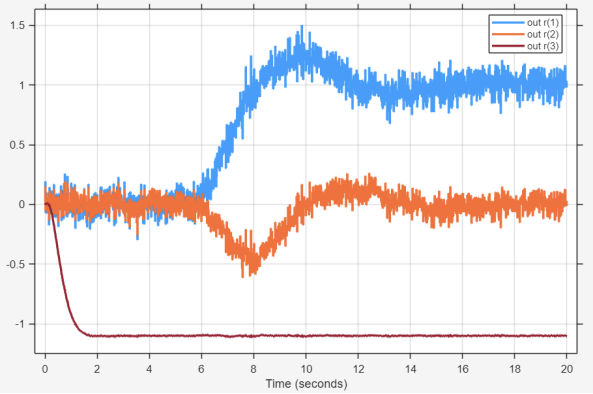


Figure 20: Discrete Model with noise - Combination of a ramp on r_1 with a initial rotation on θ_3 - Position Plot

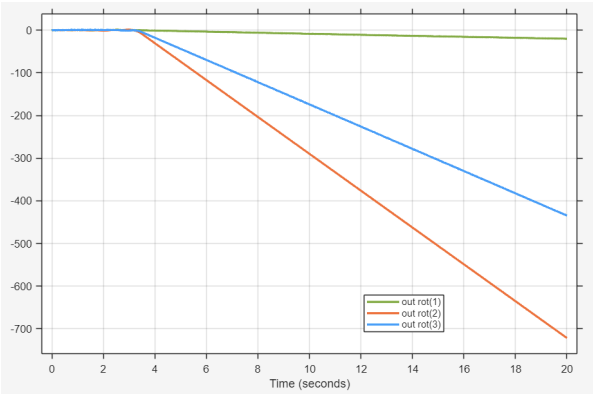


Figure 21: Continuous Model with noise - Combination of a ramp on r_1 with a initial rotation on θ_3 - Rotation Plot

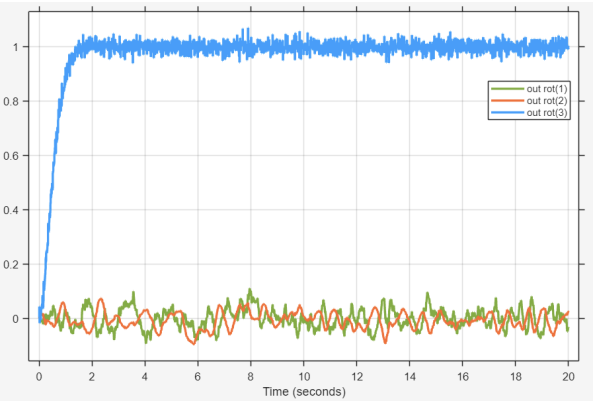


Figure 22: Discrete Model with noise - Combination of a ramp on r_1 with a initial rotation on θ_3 - Rotation Plot

Across the different scenarios, it is clear that the continuous controller senses much more noise than the discrete controller, even though Q_0 and R_0 are the same in both models. This

is a byproduct of the discretization process: the ZOH holds the previous time step for some time, effectively ignoring many deviations in short time intervals and acting similarly to a low-pass filter. Despite this being similar to a parasitic behavior, the phenomenon seems to be beneficial for the drone control. This difference is enough to stabilize the drone in the more complex reference, even after lowering the slope on r_1 on the continuous controller (see figures 19, 20, 21 and 22).

Besides the difference in the noise magnitude, no apparent differences in the model are seen.

2 Implementation on physical drone

2.1 Real-life control gains

Using the PID controllers provided in the Parrot Simulink, the group analyzed the force and moment plots to apply a constant bias to the requested forces/moments (and in turn, control actions). This is recommended to perform due to the possible physical asymmetries (including propeller/motor wear and tear) and is done to help balance the drone's take off and stabilization. An offset to compensate the gravitic force was also added in the system. Going back to wear and tear, the drone's specifications mention that the drone is able to hover at $u = 250$, and from the different field tests it is shown that would not be the case (the actual "hover value" would be closer to $u = 300$), which further dificulted the finding of a reasonable Q/R pair to control the system effectively.

When implementing the previously developed controller onto the Parrot Mambo Fly drone it immediately either became unstable or it didn't take off due to a safety mechanism system built in the drone's code, both happening due to excessive saturation on the motors. Needless to say, the control authority was too high and deviations from the reference states were heavily penalized. The immediate solution to this problem is to greatly increase R, penalizing control authority rather than state deviation.

The drone presented oscillations in the pitch angle because the state weighting matrix (Q) values for the forward movement (r_1) were too low. Consequently, the controller did not command sufficient control effort to effectively counteract external disturbances.

Ultimately, neither the standard Parrot nor the Delivery simulation environments accurately captured the true aerodynamic and physical dynamics of the hardware. Because of this reality gap, finding appropriate controller gains solely within the simulation environment proved ineffective, necessitating substantial empirical tuning.

2.2 Basic tasks

With the vertical and yaw controllers working properly one of the simplest tasks can be performed: take-off and hover.

2.2.1 Hovering

Execution date/time: 02-Jun-2026 11:54:40 Flight time: 15.53 seconds

Table 2: LQR gains used

Decoupling	$\mathbf{K_{LQR}}$	$\mathbf{k_i}$	$\mathbf{R}$
1. Vertical, r_3	$\begin{bmatrix} 1.3887 & 0.5196 \end{bmatrix}$	-1.3849	10
2. Lateral, r_2 and θ_1	$\begin{bmatrix} 0.002315 & 0.007999 & 0.002115 & 0.001603 \end{bmatrix}$	-0.001018	1000
3. Frontal, r_1 and θ_2	$\begin{bmatrix} -0.002349 & 0.008272 & -0.002159 & 0.001690 \end{bmatrix}$	0.001030	1000
4. Yaw, θ_3	$\begin{bmatrix} 0.000560 & 0.000357 \end{bmatrix}$	-0.000307	1000

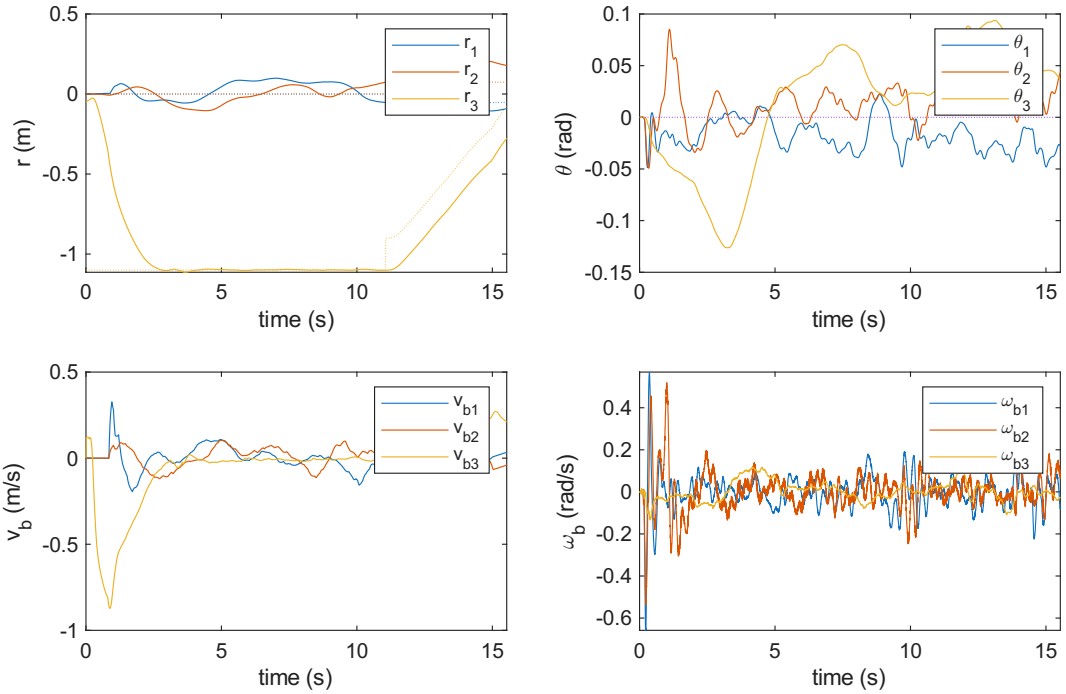


Figure 23: Laboratory Hovering task – positions, orientations and velocities

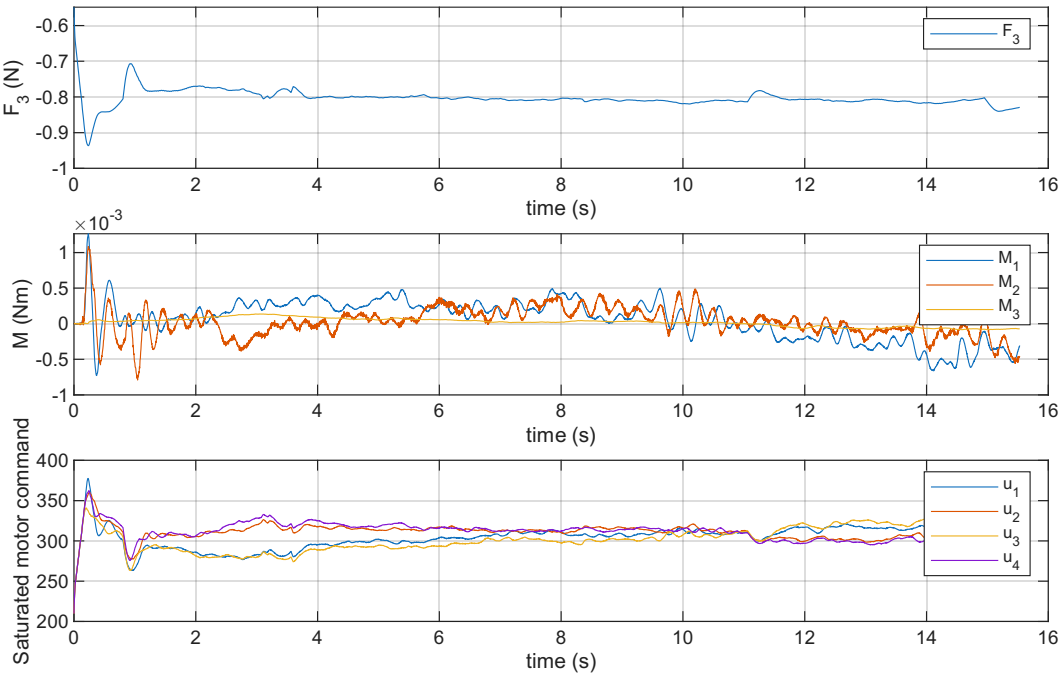


Figure 24: Laboratory Hovering task – Forces (N), Moments (Nm) and Motor Command input

For the hovering task, the simulation was successful, demonstrating that the controller gains for the first (r_1) and fourth (r_4) decoupling loops are adequately tuned to maintain steady-state flight. However, as illustrated in Figure 23, noticeable oscillations are present in the r_2 and r_3 loops. These fluctuations confirm that the remaining decoupling loops are suboptimal

and require further tuning to eliminate transient instabilities.

2.2.2 Hovering, rotating in yaw and lateral movement

Execution date/time: 02-Jun-2026 13:00:58 Flight time: 20.17 seconds

Table 3: LQR gains used (same as in 2)

Decoupling	$\mathbf{K_{LQR}}$	$\mathbf{k_i}$	$\mathbf{R}$
1. Vertical, r_3	$\begin{bmatrix}1.3887 & 0.5196\end{bmatrix}$	-1.3849	10
2. Lateral, r_2 and θ_1	$\begin{bmatrix}0.001727 & 0.008093 & 0.001750 & 0.001963\end{bmatrix}$	-0.000707	2000
3. Frontal, r_1 and θ_2	$\begin{bmatrix}-0.001757 & 0.008333 & -0.001788 & 0.002048\end{bmatrix}$	0.000718	2000
4. Yaw, θ_3	$\begin{bmatrix}0.000560 & 0.000357\end{bmatrix}$	-0.000307	1000

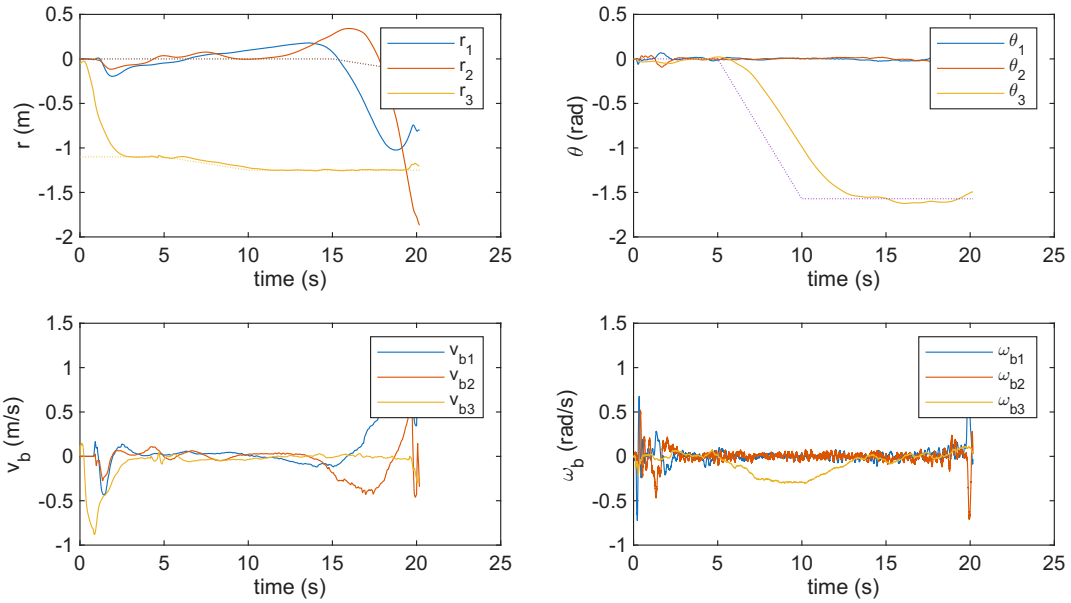


Figure 25: Laboratory Hovering task – positions, orientations and velocities

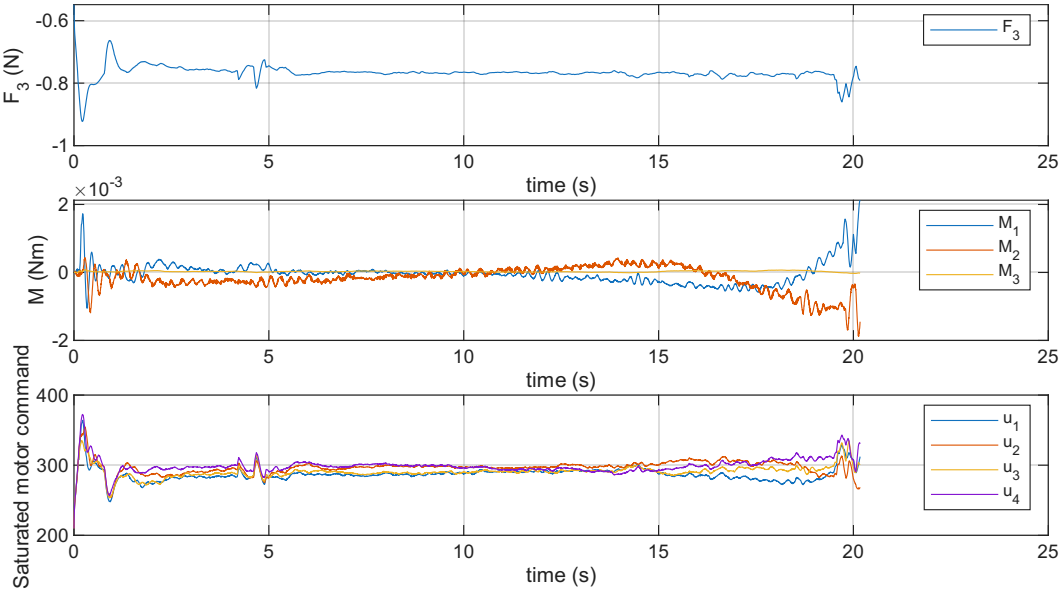


Figure 26: Laboratory Hovering task – Forces (N), Moments (Nm) and Motor Command input

The drone experienced instability during translational maneuvers in the X and Y directions right after a yaw rotation. Its was expected for the drone to move in the X positive directions and Y negative direction, instead, the exact opposite behavior occurred.

The controller failed to map the reference trajectory from the inertial frame to the drone's rotated body-fixed frame, for example, a commanded movement along the inertial Y-axis should have been translated into the drone's local X-axis after a $90°$ rotation. To rectify this, a coordinate transformation (yaw compensation) must be implemented.

Furthermore, due to suboptimally tuned controller gains, the drone exhibited excessive overshoot, failing to decelerate in time and resulting in a collision with the wall at approximately $t = 18$ s.

2.3 Trajectory planning

To implement a complex trajectory, the *Signal Editor* block was used. This block allows the reference for each position and orientation to be set to a specific value at any given time step, transitioning between sets of points using either a ramp or a step function.

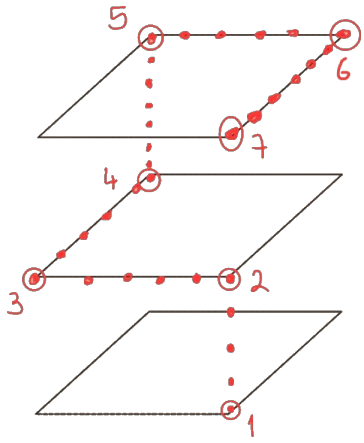


Figure 27: Trajectory's visual representation

The trajectory starts with the take-off from checkpoint 1 to 2 achieving an hovering altitude of 1.1 m. After a few seconds to allow the drone to stabilize in a hover, it translates 0.15 m along the positive X-axis from checkpoint 2 to 3, followed by a displacement in the negative Y direction from Checkpoint 3 to 4. Between checkpoints 4 and 5, a slight altitude lift of 0.1 m is executed. The drone then repeats the previous X and Y maneuvers in the opposite directions making a square when viewing from the horizontal view. Concluding the mission with a controlled landing from checkpoint 7 back to 1.

Unfortunately, this trajectory could not be validated in laboratory due to the roll and pitch not being being finely tuned. The dynamic responses obtained from the Parrot Minidrone Simulink simulation environment are presented below:

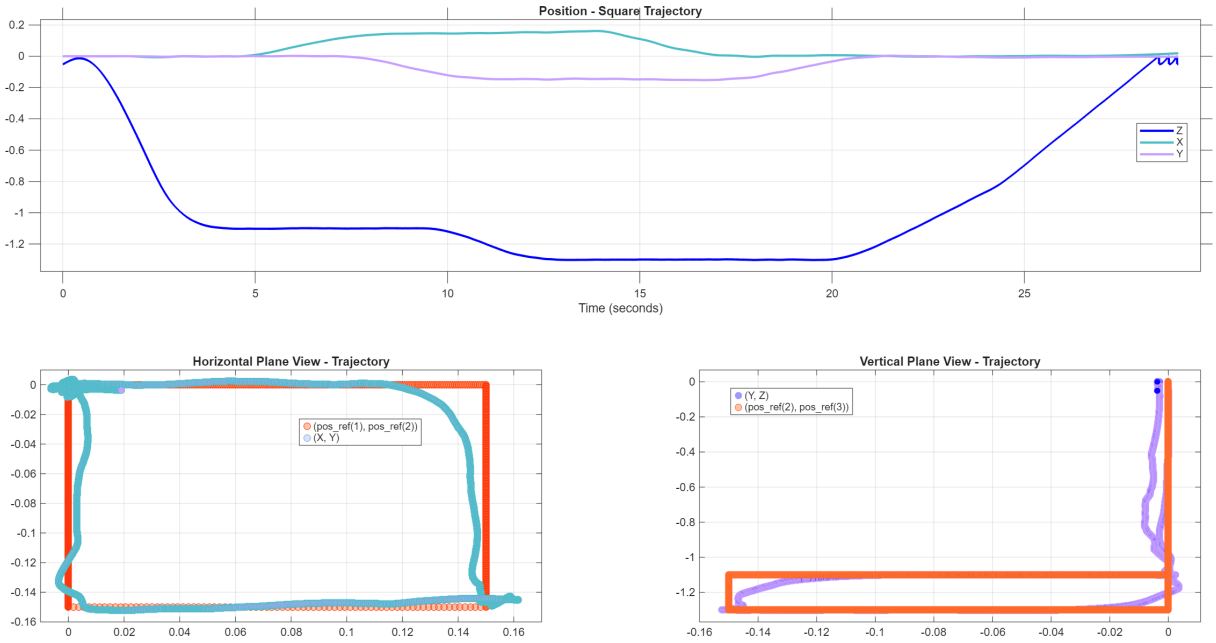


Figure 28: Parrot's Simulink r_1, r_2, r_3 and trajectory views.

Even with good results this simulation is not reliable due to several unmodeled dynamics. Specifically, non-linear aerodynamic perturbations and the true physical mass variations of the drone were not implemented within the Simulink model.